LeetCode Practice

40 Medium: Heap

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

40
MEDIUM

Patterns (in order)

1. Heap (40)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

40 questions | tap a title to jump to it

Medium questions (40)		Go >
Heap (40)		**Go >**
M	1. Kth Largest Element in an Array	LC 215
M	2. Meeting Rooms II	LC 253
M	3. Ugly Number II	LC 264
M	4. Top K Frequent Elements	LC 347
M	5. Design Twitter	LC 355
M	6. Find K Pairs with Smallest Sums	LC 373
M	7. Kth Smallest Element in a Sorted Matrix	LC 378
M	8. Sort Characters By Frequency	LC 451
M	9. The Maze II	LC 505
M	10. Task Scheduler	LC 621
M	11. Find K Closest Elements	LC 658
M	12. Split Array into Consecutive Subsequences	LC 659
M	13. Network Delay Time	LC 743
M	14. Reorganize String	LC 767
M	15. Cheapest Flights Within K Stops	LC 787
M	16. Sort an Array	LC 912
M	17. K Closest Points to Origin	LC 973
M	18. Campus Bikes	LC 1057
M	19. Car Pooling	LC 1094
M	20. Path With Maximum Minimum Value	LC 1102
M	21. Connecting Cities With Minimum Cost	LC 1135
M	22. Minimum Cost to Connect Sticks	LC 1167
M	23. Maximum Number of Events That Can Be Attended	LC 1353
M	24. Longest Happy String	LC 1405
M	25. Diagonal Traverse II	LC 1424
M	26. Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit	LC 1438
M	27. Avoid Flood in The City	LC 1488
M	28. Path with Maximum Probability	LC 1514
M	29. Path with Minimum Effort	LC 1631
M	30. Furthest Building You Can Reach	LC 1642
M	31. Jump Game VI	LC 1696
M	32. Single-Threaded CPU	LC 1834
M	33. Process Tasks Using Servers	LC 1882
M	34. Find the Kth Largest Integer in the Array	LC 1985
M	35. Stock Price Fluctuation	LC 2034
M	36. Two Best Non-Overlapping Events	LC 2054
M	37. Design a Food Rating System	LC 2353
M	38. Continuous Subarrays	LC 2762
M	39. Find Minimum Time to Reach Last Room I	LC 3341
M	40. Minimum Cost Path with Edge Reversals	LC 3650

Medium questions (40)

MEDIUM | TOPIC 1 OF 1

Heap

40 Medium questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Kth Largest Element in an Array LC 215
2. Meeting Rooms II LC 253
3. Ugly Number II LC 264
4. Top K Frequent Elements LC 347
5. Design Twitter LC 355
6. Find K Pairs with Smallest Sums LC 373
7. Kth Smallest Element in a Sorted Matrix LC 378
8. Sort Characters By Frequency LC 451
9. The Maze II LC 505
10. Task Scheduler LC 621
11. Find K Closest Elements LC 658
12. Split Array into Consecutive Subsequences LC 659
13. Network Delay Time LC 743
14. Reorganize String LC 767
15. Cheapest Flights Within K Stops LC 787
16. Sort an Array LC 912
17. K Closest Points to Origin LC 973
18. Campus Bikes LC 1057
19. Car Pooling LC 1094
20. Path With Maximum Minimum Value LC 1102
21. Connecting Cities With Minimum Cost LC 1135
22. Minimum Cost to Connect Sticks LC 1167
23. Maximum Number of Events That Can Be Attended LC 1353
24. Longest Happy String LC 1405
25. Diagonal Traverse II LC 1424
26. Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit LC 1438
27. Avoid Flood in The City LC 1488
28. Path with Maximum Probability LC 1514
29. Path with Minimum Effort LC 1631
30. Furthest Building You Can Reach LC 1642
31. Jump Game VI LC 1696
32. Single-Threaded CPU LC 1834
33. Process Tasks Using Servers LC 1882
34. Find the Kth Largest Integer in the Array LC 1985
35. Stock Price Fluctuation LC 2034
36. Two Best Non-Overlapping Events LC 2054
37. Design a Food Rating System LC 2353
38. Continuous Subarrays LC 2762
39. Find Minimum Time to Reach Last Room I LC 3341
40. Minimum Cost Path with Edge Reversals LC 3650

MEDIUM | Heap | LeetCode #215 | 1 of 40 | 1/40

Kth Largest Element in an Array

QUESTION

Given an integer array nums and an integer k, return the kth largest element in the array.

Note that it is the kth largest element in the sorted order, not the kth distinct element.

Can you solve it without sorting?

Example 1	
Input	`nums = [3,2,1,5,6,4], k = 2`
Output	`5`

Example 2	
Input	`nums = [3,2,3,1,2,4,5,5,6], k = 4`
Output	`4`

CONSTRAINTS

- $1 <= k <= nums.length <= 10^5$
- $10^4 <= nums[i] <= 10^4$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the kth largest element in an unsorted array.

Not kth distinct — duplicates count toward the position. Right?"

"A few quick questions:

k is always valid (1 <= k <= n)? Yes. Can elements be negative? Yes."

"Let me walk: nums=[3,2,1,5,6,4], k=2 -> sorted desc=[6,5,4,3,2,1] -> 5 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
sort nums in descending order
return the element at index k - 1
```

TIME	SPACE
O(n log n)	O(1)

WHY IT IS SLOW

Sorts everything to read one position. A min-heap of size k is O(n log k); quickselect is O(n) on average.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is sorting all n elements when we only need the top k.

Min-heap of size k: maintain the k largest elements seen so far.

The heap's root (minimum of the k largest) is the kth largest.

Push each element; if heap grows beyond k, pop the smallest.

Time: O(n log k). Space: O(k).

Better than O(n log n) when k is much smaller than n.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
import heapq
class _215_KthLargestElement:
    def findKthLargest(self, nums, k):
        min_heap = []
        for num in nums:
            heapq.heappush(min_heap, num)
            if len(min_heap) > k:
                heapq.heappop(min_heap)
        return min_heap[0]
```

PHASE 5 TEST & DEBUG

"Let me trace findKthLargest([3,2,1,5,6,4], k=2)."

Min-heap of size k. Maintain k largest seen.

Push 3. Push 2. Push 1. Heap=[1,2,3]. Push 5: 5>heap[0]=1 -> pop 1, push 5. Heap=[2,3,5].

Push 6: 6>2 -> pop 2, push 6. Heap=[3,5,6]. Push 4: 4>3 -> pop 3, push 4. Heap=[4,5,6].

Top of min-heap = 4 = 2nd largest ok

Alt: QuickSelect. Partition around pivot. [3,2,1,5,6,4], k=2 (want index n-k=4).

Partition: pivot=4. Left[<4]=[3,2,1], right[>4]=[5,6]. Pivot at index 3. n-k=4>3 -> search right.

Right=[5,6]. Pivot=6. Left=[5]. 5 at index 4 = target ok. Return 5 ok

WHY k-size min-heap: anything smaller than the current kth largest is useless — the heap minimum IS the kth largest at all times, O(n log k) total.

PHASE 6 COMPLEXITY & FOLLOW-UP

Min-heap: O(n log k). Sort: O(n log n). Heap wins when k << n.

Quickselect: O(n) average, $O(n^2)$ worst — optimal for single large-n queries.

Follow-up: Kth Smallest in Sorted Matrix (#378) — 2D extension using a heap.

Follow-up: streaming kth largest (#703) — maintain a min-heap of size k permanently."

Space: O(n) — proportional to input or output size; O(1) extra if in-place.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Heap | LeetCode #253 | 2 of 40 | 2/40

Meeting Rooms II

QUESTION

Given an array of meeting time intervals intervals where intervals[i] = [$start_i$, end_i], return the minimum number of conference rooms required.

Example 1	
Input	`intervals = [[0,30],[5,10],[15,20]]`
Output	`2`

Example 2	
Input	`intervals = [[7,10],[2,4]]`
Output	`1`

CONSTRAINTS

- 1 <= intervals.length <= 10^4
- 0 <= $start_i$ < end_i <= 10^6

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given meeting time intervals, find the minimum number of conference rooms required.

Two meetings can share a room only if one ends before or when the other starts. Right?"

"A few quick questions:

Do meetings that touch (end == start) overlap? No — they can share a room.

Return the minimum rooms, not a specific assignment?"

"Let me walk: [[0,30],[5,10],[15,20]] -> [0,30] overlaps with [5,10] -> 2 rooms ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
rooms = []
for start in starts:
    find any room whose last meeting has already
        ended
    if one exists:
        put the meeting there, else open a new room
return len(rooms)
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Searches every room for each meeting. A min-heap of room end times (pop the earliest end if it is free, then push the new end) is O(n log n); sorting starts and ends separately and sweeping is also O(n log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n^2) pairwise overlap check.

Min-heap of end times: tracks when each occupied room becomes free.

Sort meetings by start time. For each meeting:

- If the earliest-ending room (heap top) finishes before or at the meeting start, reuse it.

- Otherwise, allocate a new room.

Heap size at the end = minimum rooms needed.

Time: O(n log n). Space: O(n) for the heap."

Key insight: the min-heap always exposes the earliest-ending room — if it ends before the next meeting starts, reuse it; otherwise allocate a new room.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
import heapq
class _253_MeetingRoomsII:
    def minMeetingRooms(self, intervals):
        if not intervals: return 0
        intervals.sort(key=lambda x: x[0])
        end_times = []
        for start, end in intervals:
            if end_times and end_times[0] <= start:
                heapq.heapreplace(end_times, end)
            else:
                heapq.heappush(end_times, end)
        return len(end_times)
```

PHASE 5 TEST & DEBUG

"Let me trace how the heap property guides which element comes next."

[[0,30],[5,10],[15,20]] sorted=same:

[0,30]: heap=[] -> push 30 -> [30].

[5,10]: heap[0]=30 > 5 -> push 10 -> [10,30].

[15,20]: heap[0]=10 <= 15 -> replace 10 with 20 -> [20,30]. len=2 ok

[[7,10],[2,4]] sorted=[[2,4],[7,10]]:

[2,4]: push 4 -> [4]. [7,10]: 4<=7 -> replace with 10 -> [10]. len=1 ok

Heapreplace is equivalent to pop+push but in one O(log n) operation

PHASE 5 TEST & DEBUG

WHY min-heap of end times: the earliest-ending room is the best candidate to reuse for the next meeting — greedy reuse minimises new room allocations.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n)	**O(n)**
sort + n heap ops	heap holds at most n end times

Alternative: split into sorted start/end events, use two pointers — same complexity, no heap.

Follow-up: Meeting Rooms I (#252) — just check if any two overlap (sort by start).

Follow-up: minimum cost scheduling — extend with room priorities or weighted meetings.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Heap | LeetCode #264 | 3 of 40 | 3/40

Ugly Number II

QUESTION

An ugly number is a positive integer whose prime factors are limited to 2, 3, and 5.

Given an integer n, return the nth ugly number.

Example 1	
Input	`n = 10`
Output	`12`
Explanation	*[1, 2, 3, 4, 5, 6, 8, 9, 10, 12] is the sequence of the first 10 ugly numbers.*

Example 2	
Input	`n = 1`
Output	`1`
Explanation	*1 has no prime factors, therefore all of its prime factors are limited to 2, 3, and 5.*

CONSTRAINTS

- 1 <= n <= 1690

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the nth ugly number (positive integer with only prime factors 2, 3, 5).

1 is the first ugly number. Right?"

"A few quick questions:

Generate them in order? Yes. n=1 -> 1?"

"Let me walk: first 10 ugly numbers: 1,2,3,4,5,6,8,9,10,12. n=10 -> 12 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
count = 0
for x = 1, 2, 3 ...:
    if x is ugly (strip 2s, 3s and 5s leaves 1):
        count = count + 1
    if count == n:
        break
```

TIME	SPACE
O(m log m)	O(1)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

m is the value of the n-th ugly number, far larger than n (about 10^9 for n = 1690). Three pointers that build ugly numbers in order (next = min(2a, 3b, 5c)) are O(n) time and O(n) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Min-heap or three-pointer DP — O(n log n) or O(n). Walk-through (three-pointer):"

`ugly = [1]. i2=i3=i5=0 (pointers into ugly array for ×2, ×3, ×5 candidates).`

For k in 1..n:

`next2 = ugly[i2]*2. next3 = ugly[i3]*3. next5 = ugly[i5]*5.`

`nxt = min(next2, next3, next5).`

`ugly.append(nxt).`

If nxt == next2: i2++.

If nxt == next3: i3++.

If nxt == next5: i5++.

Return ugly[n-1].

WHY advance all equal pointers: 6=2×3=3×2 would appear as next2 AND next3 -> advance both

to avoid duplicates.

WHY three pointers: each pointer tracks which ugly number to multiply by 2/3/5 next.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _264_UglyNumberII:
    def nthUglyNumber(self, n):
        ugly = [1]
        i2 = i3 = i5 = 0
        while len(ugly) < n:
            next2, next3, next5 = ugly[i2]*2, ugly[i3]*3,
ugly[i5]*5
            nxt = min(next2, next3, next5)
            ugly.append(nxt)
            if nxt == next2: i2 += 1
            if nxt == next3: i3 += 1
            if nxt == next5: i5 += 1
        return ugly[-1]
```

PHASE 5 TEST & DEBUG

"Let me trace nthUglyNumber(10)."

Pointers p2=p3=p5=0. ugly=[1].

Next ugly = min(ugly[p2]*2, ugly[p3]*3, ugly[p5]*5)=min(2,3,5)=2. Advance p2. ugly=[1,2].

`min(4,3,5)=3. Advance p3. ugly=[1,2,3].`

`min(4,6,5)=4. Advance p2. ugly=[1,2,3,4].`

`min(6,6,5)=5. Advance p5. ugly=[1,2,3,4,5].`

PHASE 5 TEST & DEBUG

```
min(6,6,10)=6. Both p2 and p3 give 6 -> advance both.
  ugly=[1,2,3,4,5,6].
min(8,9,10)=8. ugly=[1,2,3,4,5,6,8]... Continue to
  10th: [1,2,3,4,5,6,8,9,10,12]. Return 12 ok
```

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)** for the sequence

Follow-up: Super Ugly Number (#313) — arbitrary prime set; k pointers instead of 3.

Follow-up: Ugly Number I (#263) — check if a single number is ugly.

MEDIUM | Heap | LeetCode #347 | 4 of 40 | 4/40

Top K Frequent Elements

QUESTION

Given an integer array nums and an integer k, return the k most frequent elements. You may return the answer in any order.

Example 1	
Input	`nums = [1,1,1,2,2,3], k = 2`
Output	`[1,2]`

Example 2	
Input	`nums = [1], k = 1`
Output	`[1]`

Example 3	
Input	`nums = [1,2,1,2,1,2,3,1,3,2], k = 2`
Output	`[1,2]`

CONSTRAINTS

- $1 <= nums.length <= 10^5$
- $10^4 <= nums[i] <= 10^4$
- k is in the range [1, the number of unique elements in the array].
- It is guaranteed that the answer is unique.
- Follow up: Your algorithm's time complexity must be better than O(n log n), where n is the array's size.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return the k most frequent elements. The answer is unique (no ties at position k).

Return in any order. Must be better than O(n log n). Right?"

"A few quick questions:

Can k equal the number of unique elements? Yes. Values can be negative? Yes."

"Let me walk: nums=[1,1,1,2,2,3], k=2 -> 1(freq=3), 2(freq=2) -> [1,2] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
count how often each number occurs
sort the distinct numbers by count
return the first k
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n log n)	O(n)

WHY IT IS SLOW

Sorts all distinct values to keep k. A min-heap of size k is O(n log k); bucketing numbers by frequency (index = count) is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n log n) sort of all frequencies.

Bucket sort: create n+1 buckets indexed by frequency.

Each bucket at index f holds all numbers that appear exactly f times.

Scan buckets from index n down to 0, collect until we have k elements.

Time: O(n). Space: O(n).

Does that make sense before I code it?"

Key insight: the hash map provides O(1) lookup — a single pass over the data replaces the O(n) inner scan, reducing total complexity by an order of magnitude.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _347_TopKFrequent:
    def topKFrequent(self, nums, k):
        from collections import Counter
        count = Counter(nums)
        buckets = [[] for _ in range(len(nums) + 1)]
        for num, freq in count.items():
            buckets[freq].append(num)
        result = []
        for freq in range(len(buckets) - 1, 0, -1):
            for num in buckets[freq]:
                result.append(num)
                if len(result) == k:
                    return result
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace topKFrequent([1,1,1,2,2,3], k=2)."

Count: {1:3,2:2,3:1}. Min-heap of size k.

Push (3,1). Heap=[(3,1)]. Push (2,2): heap=[(2,2),(3,1)]. Push (1,3): heap min=2>1 -> no push (heap full AND new<min -> skip). Wait: push all then keep k largest.

Actually: for each (freq, val): if heap size < k -> push. Else if freq > heap[0][0] -> pop, push.

After (3,1): heap=[(3,1)]. After (2,2): heap=[(2,2),(3,1)]. After (1,3): 1<heap[0][0]=2 -> skip.

Return [val for freq,val in heap] = [1,2] ok

Bucket sort alt: buckets[freq].append(val). Iterate from high freq down, collect k. O(n).

PHASE 5 TEST & DEBUG

WHY k-size min-heap: we evict the least frequent when the heap exceeds k — the surviving k elements ARE the top-k frequent, O(n log k) vs O(n log n) full sort.

PHASE 6 COMPLEXITY & FOLLOW-UP

Bucket sort: O(n) time, O(n) space. Heap approach: O(n log k) — worse but simpler.

Bucket sort is the optimal O(n) solution when frequencies <= n (always true here).

Follow-up: Top K Frequent Words (#692) — break frequency ties alphabetically.

Follow-up: streaming top-k — maintain a min-heap of size k as elements arrive.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Heap | LeetCode #355 | 5 of 40 | 5/40

Design Twitter

QUESTION

Design a simplified version of Twitter where users can post tweets, follow/unfollow another user, and is able to see the 10 most recent tweets in the user's news feed.

Implement the Twitter class:

CONSTRAINTS

- Twitter() Initializes your twitter object.
- void postTweet(int userId, int tweetId) Composes a new tweet with ID tweetId by the user userId. Each call to this function will be made with a unique tweetId.
- List<Integer> getNewsFeed(int userId) Retrieves the 10 most recent tweet IDs in the user's news feed. Each item in the news feed must be posted by users who the user followed or by the user themself. Tweets must be ordered from most recent to least recent.
- void follow(int followerId, int followeeId) The user with ID followerId started following the user with ID followeeId.
- void unfollow(int followerId, int followeeId) The user with ID followerId started unfollowing the user with ID followeeId.
- 1 <= userId, followerId, followeeId <= 500
- 0 <= tweetId <= 10^4
- All the tweets have unique IDs.
- At most 3 * 10^4 calls will be made to postTweet, getNewsFeed, follow, and unfollow.
- A user cannot follow himself.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Design a simplified Twitter: postTweet, getNewsFeed (10 most recent from self+followees), follow, unfollow. Right?"

"A few quick questions:

getNewsFeed needs tweets ordered most-recent-first across all followed users? Yes.

A user follows themselves implicitly (sees own tweets)? Yes."

"Let me walk: post(1,5), getNewsFeed(1)->[5]. follow(1,2), post(2,6), getNewsFeed(1)->[6,5] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
keep one list of every tweet (user, id, time)
def getNewsFeed(u):
    scan all tweets, keep those by u or u's
        followees, take the 10 newest
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(T) per feed	O(T + F)

WHY IT IS SLOW

Scans every tweet in the system per request. Store each user's tweets separately and merge the followees' newest 10 with a max-heap: O(F log F + 10 log F).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Heap-based tweet feed + follow/unfollow maps — O(log U) per getNewsFeed. Walk-through:"

`follow_map[userId] = set of followees.`

`tweet_map[userId] = list of (timestamp, tweetId) in FIFO order (append order = timestamp order).`

postTweet: append (global_time++, tweetId) to tweet_map[userId].

`getNewsFeed(userId):`

min-heap of size 10 (negate timestamp for max-heap behavior).

For each user in follow_map[userId] {userId}:

Add last tweet (tweet_map[user][-1]) to heap (with pointer to position in their list).

Extract top-10: pop max-timestamp tweet, add to result; if user has more tweets, push next.

follow/unfollow: add/remove from follow_map[userId].

WHY heap: efficiently merge multiple tweet lists to get top-10 globally.

Time: O(U log U) per getNewsFeed where U = followees. Space: O(total tweets).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
import heapq
from collections import defaultdict
class _355_DesignTwitter:
    def __init__(self):
        self.timestamp = 0
        self.user_tweets = defaultdict(list)
        self.user_follows = defaultdict(set)

    def postTweet(self, userId, tweetId):
        self.user_tweets[userId].append((self.timestamp, t
weetId))
        self.timestamp += 1

    def getNewsFeed(self, userId):
        heap = []
        followees = self.user_follows[userId] | {userId}
        for uid in followees:
            tweets = self.user_tweets[uid]
            if tweets:
                idx = len(tweets) - 1
                time, tweetId = tweets[idx]
                heapq.heappush(heap, (-time, tweetId, uid,
```

```
 idx - 1))
        result = []
        while heap and len(result) < 10:
            neg_time, tweetId, uid, idx = heapq.heappop(he
ap)
            result.append(tweetId)
            if idx >= 0:
                time, tid = self.user_tweets[uid][idx]
                heapq.heappush(heap, (-time, tid, uid, idx
 - 1))
        return result

    def follow(self, followerId, followeeId):
        if followerId != followeeId:
            self.user_follows[followerId].add(followeeId)

    def unfollow(self, followerId, followeeId):
        self.user_follows[followerId].discard(followeeId)
```

PHASE 5 TEST & DEBUG

"Let me trace Twitter: follow(1,2), postTweet(1,5), getNewsFeed(1), unfollow(1,2), getNewsFeed(1)."

userId->set of followees. userId->list of (timestamp,tweetId).

follow(1,2): following[1]={2}. postTweet(1,5): tweets[1]=[(0,5)].

getNewsFeed(1): self + followees = {1,2}. Merge tweet lists by timestamp desc.

tweets[1]=[(0,5)], tweets[2]=[]. Top 10 -> [(5)] ok. Return [5] ok

unfollow(1,2): following[1]={}.

getNewsFeed(1): only {1}'s tweets -> [5] ok (own tweet still there).

WHY heap merge: n users × up to 10 tweets each -> min-heap of size n for O(n log n) merge.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(F log F) per feed	**O(n + F)**

Follow-up: Merge k Sorted Lists (#23) — identical heap-merge pattern.

Follow-up: support retweets or likes — extend tweet tuple with metadata.

IN YOUR APP | Expose | Community

MEDIUM | Heap | LeetCode #373 | 6 of 40 | 6/40

Find K Pairs with Smallest Sums

QUESTION

You are given two integer arrays nums1 and nums2 sorted in non-decreasing order and an integer k.

Define a pair (u, v) which consists of one element from the first array and one element from the second array.

Return the k pairs $(u_1, v_1), (u_2, v_2), ..., (u_k, v_k)$ with the smallest sums.

Example 1	
Input	`nums1 = [1,7,11], nums2 = [2,4,6], k = 3`
Output	`[[1,2],[1,4],[1,6]]`
Explanation	*The first 3 pairs are returned from the sequence: [1,2],[1,4],[1,6],[7,2],[7,4],[11,2],[7,6],[11,4],[11,6]*

Example 2	
Input	`nums1 = [1,1,2], nums2 = [1,2,3], k = 2`
Output	`[[1,1],[1,1]]`
Explanation	*The first 2 pairs are returned from the sequence: [1,1],[1,1],[1,2],[2,1],[1,2],[2,2],[1,3],[1,3],[2,3]*

CONSTRAINTS

- 1 <= nums1.length, nums2.length <= 10^5
- 10^9 <= nums1[i], nums2[i] <= 10^9
- nums1 and nums2 both are sorted in non-decreasing order.
- 1 <= k <= 10^4
- k <= nums1.length * nums2.length

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given two sorted arrays, return the k pairs (u,v) with the smallest sums.

A pair is (nums1[i], nums2[j]). Right?"

"A few quick questions:

k pairs total from the cross product? Yes."

"Let me walk: nums1=[1,7,11], nums2=[2,4,6], k=3 -> [[1,2],[1,4],[1,6]] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
pairs = every (x, y) with x from the first array and
    y from the second
sort the pairs by x + y
return the first k
```

TIME	SPACE
O(m * n * log(m * n))	O(m * n)

WHY IT IS SLOW

Builds and sorts all m * n pairs. A min-heap seeded with (nums1[i], nums2[0]) for the first k values of nums1, advancing the second index when popped, is O(k log k).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Min-heap of (sum, i, j) expanding in sorted order — O(k log k). Walk-through:"

Sort nums1 and nums2.

Heap: push (nums1[0]+nums2[0], 0, 0). visited={(0,0)}.

For _ in range(k):

(s, i, j) = heappop. Append [nums1[i], nums2[j]] to result.

For (ni, nj) in [(i+1,j), (i,j+1)]:

If (ni,nj) not in visited and ni<n1 and nj<n2:

`heappush(s+nums1[ni]+nums2[nj]-... wait, push (nums1[ni]+nums2[nj], ni, nj)).`

`visited.add((ni,nj)).`

Return result.

WHY sort first: smallest sums come from (0,0); next candidates are (1,0) and (0,1).

WHY heap: greedily extracts the globally smallest sum among all unvisited pairs.

WHY visited set: each (i,j) pair is added at most once.

Time: O(k log k). Space: O(k).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
import heapq
class _373_FindKPairsSmallestSums:
    def kSmallestPairs(self, nums1, nums2, k):
        if not nums1 or not nums2:
            return []
        heap = [(nums1[i] + nums2[0], i, 0) for i in range
(min(k, len(nums1)))]
        heapq.heapify(heap)
        result = []
        while heap and len(result) < k:
            s, i, j = heapq.heappop(heap)
            result.append([nums1[i], nums2[j]])
            if j + 1 < len(nums2):
```

```
                heapq.heappush(heap, (nums1[i] + nums2[j+1
], i, j+1))
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace kSmallestPairs([1,7,11],[2,4,6], k=3)."

Min-heap with (sum, i, j): start with [(1+2,0,0)]=[(3,0,0)]

Pop (3,0,0)->pair [1,2]. Push (1+4,0,1)=(5,0,1) and (7+2,1,0)=(9,1,0). Heap=[(5,0,1),(9,1,0)]

Pop (5,0,1)->pair [1,4]. Push (1+6,0,2)=(7,0,2). Heap=[(7,0,2),(9,1,0)]

Pop (7,0,2)->pair [1,6]. k=3 reached. Result=[[1,2],[1,4],[1,6]] ok

WHY heap: always get next smallest pair; push right and down neighbors.

Only push (i+1,j) and (i,j+1) to avoid duplicates.

Only first column initially pushed to avoid duplicates; push right neighbor on each pop.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(k log k)	**O(k)** heap

Follow-up: Kth Smallest in Sorted Matrix (#378) — same heap pattern on a 2D matrix.

Follow-up: if multiple queries with different k, precompute with a priority queue.

MEDIUM | Heap | LeetCode #378 | 7 of 40 | 7/40

Kth Smallest Element in a Sorted Matrix

QUESTION

Given an n x n matrix where each of the rows and columns is sorted in ascending order, return the kth smallest element in the matrix.

Note that it is the kth smallest element in the sorted order, not the kth distinct element.

You must find a solution with a memory complexity better than $O(n^2)$.

Example 1

Input	`matrix = [[1,5,9],[10,11,13],[12,13,15]], k = 8`
Output	`13`
Explanation	*The elements in the matrix are [1,5,9,10,11,12,13, 13 ,15], and the 8th smallest number is 13*

Example 2

Input	`matrix = [[-5]], k = 1`
Output	`-5`

CONSTRAINTS

- n == matrix.length == matrix[i].length
- 1 <= n <= 300
- 10^9 <= matrix[i][j] <= 10^9
- All the rows and columns of matrix are guaranteed to be sorted in non-decreasing order.
- 1 <= k <= n^2
- Follow up:
- Could you solve the problem with a constant memory (i.e., O(1) memory complexity)?
- Could you solve the problem in O(n) time complexity? The solution may be too advanced for an interview but you may find reading this paper fun.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the kth smallest element in an n×n matrix where rows and columns are sorted. Right?"

"A few quick questions:

Matrix is not fully sorted (like #74) — rows and columns sorted independently? Yes."

"Let me walk: matrix=[[1,5,9],[10,11,13],[12,13,15]], k=8 -> 13 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
put every matrix element in a list
sort the list
return the element at index k - 1
```

TIME	SPACE
O(n^2 log n)	O(n^2)

WHY IT IS SLOW

Sorts all n^2 elements. A min-heap seeded with the first column, popped k times, is O(k log n); binary search on the value with a staircase count is O(n log(max - min)).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is O(k log n).

Binary search on value range: lo=matrix[0][0], hi=matrix[n-1][n-1].

Count elements <= mid using staircase search.

Find smallest value where count >= k.

Time: O(n log(max-min)). Space: O(1)."

Key insight: binary search on the VALUE range [matrix[0][0], matrix[-1][-1]] — a staircase count of elements <= mid (O(n) per check) makes feasibility testing efficient; O(n log(max-min)) total.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _378_KthSmallestElementSortedMatrix:
    def kthSmallest(self, matrix, k):
        n = len(matrix)
        def count_le(mid):
            count = 0
            r, c = n-1, 0
            while r >= 0 and c < n:
                if matrix[r][c] <= mid:
                    count += r + 1; c += 1
                else:
                    r -= 1
            return count
        lo, hi = matrix[0][0], matrix[n-1][n-1]
        while lo < hi:
            mid = (lo + hi) // 2
            if count_le(mid) >= k:
                hi = mid
            else:
                lo = mid + 1
        return lo
```

PHASE 5 TEST & DEBUG

"Let me trace kthSmallest on matrix=[[1,5,9],[10,11,13],[12,13,15]], k=8."

Min-heap approach: push (matrix[0][i], 0, i) for each col

Pop (1,0,0)->push (10,1,0). Pop(5,0,1)->push(11,1,1). Pop(9,0,2)->push(13,1,2).

Pop(10,1,0)->push(12,2,0). Pop(11,1,1)->push(13,2,1). Pop(12,2,0)->push(no row 3).

Pop(13,1,2)->push(15,2,2). k=8: Pop(13,2,1)->return 13 ok

Binary search approach: lo=1,hi=15,mid=8: count<=8 in matrix=4<8->lo=9; mid=12: count=6<8->lo=13; return 13 ok

Binary search finds smallest val v where count(<=v) >= k; count() is O(n+m).

Heap approach: O(k log n) if k is small; binary search approach O((m+n) log(max*n)) for any k.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log(max-min))	**O(1)**

Heap approach: O(k log n) — better when k is small.

Follow-up: Find K Pairs with Smallest Sums (#373) — similar heap approach.

Follow-up: if matrix is full-sorted (#74), treat as 1D array.

MEDIUM | Heap | LeetCode #451 | 8 of 40 | 8/40

Sort Characters By Frequency

QUESTION

Given a string s, sort it in decreasing order based on the frequency of the characters. The frequency of a character is the number of times it appears in the string.

Return the sorted string. If there are multiple answers, return any of them.

Example 1

Input	`s = "tree"`
Output	`"eert"`
Explanation	*'e' appears twice while 'r' and 't' both appear once. So 'e' must appear before both 'r' and 't'. Therefore "eetr" is also a valid answer.*

Example 2

Input	`s = "cccaaa"`
Output	`"aaaccc"`
Explanation	*Both 'c' and 'a' appear three times, so both "cccaaa" and "aaaccc" are valid answers. Note that "cacaca" is incorrect, as the same characters must be together.*

Example 3

Input	`s = "Aabb"`
Output	`"bbAa"`
Explanation	*"bbaA" is also a valid answer, but "Aabb" is incorrect. Note that 'A' and 'a' are treated as two different characters.*

CONSTRAINTS

- $1 <= s.length <= 5 * 10^5$
- s consists of uppercase and lowercase English letters and digits.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Sort characters by decreasing frequency. If same frequency, any order. Right?"

"A few quick questions:

Return any valid ordering? Yes."

"Let me walk: s='tree' -> 'e' appears 2 times, 't','r' once -> 'eert' or 'eetr' ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for character in characters:
    count its occurrences by scanning the string
    sort the characters by count
    write each character count times
```

TIME	SPACE
O(n * k)	O(n)

WHY IT IS SLOW

A scan per distinct character (k of them). Count all characters in one pass with a hash map, then order by count (sort, or buckets indexed by count) in O(n) or O(n log k).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Count frequencies, sort by (-freq, char) — O(n log n). Walk-through:"

`freq = Counter(s).`

Sort by (-frequency, character). <- descending frequency; ties broken by char (arbitrary here)

".join(c * freq[c] for c, _ in sorted_items).

Alternative: bucket sort (frequency as bucket index) -> O(n):

buckets[freq].append(char). Iterate buckets from high to low.

Append char * freq to result.

WHY repeat char: 'aaabbc' -> 'a' appears 3 times: 'aaa'. 'b' twice: 'bb'. Etc.

WHY sort by frequency descending: higher-frequency chars come first in output.

Time: O(n log n) sort or O(n) bucket. Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
from collections import Counter
class _451_SortCharactersByFrequency:
    def frequencySort(self, s):
        count = Counter(s)
        max_freq = max(count.values())
        buckets = [[] for _ in range(max_freq + 1)]
        for ch, freq in count.items():
            buckets[freq].append(ch)
        result = []
        for freq in range(max_freq, 0, -1):
            for ch in buckets[freq]:
                result.append(ch * freq)
        return ''.join(result)
```

PHASE 5 TEST & DEBUG

"Let me trace frequencySort on s='tree'."

count: t:1, r:1, e:2

Bucket by freq: bucket[1]=['t','r'], bucket[2]=['e']

Build result from high freq to low: 'ee'+'tr' (or 'ee'+'rt') = 'eetr' ok

Trace s='cccaaa': count: c:3,a:3. bucket[3]=['c','a']. Result 'cccaaa' or 'aaaccc' ok

WHY bucket sort: O(n) vs O(n log n) heap; freq bounded by string length.

Edge: all same char -> single bucket; result same as input.

Result: 'eetr' or 'eert' (e:2, then t:1, r:1 in any order). Stability by insertion order.

Trace: s="tree" -> count: {t:1,r:1,e:2} -> sort by freq desc: e(2),t(1),r(1) -> result="eetr" ok. Tie-breaking: "t" vs "r" by any consistent order.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
count + bucket + output	

Follow-up: Top K Frequent Words (#692) — same frequency sort with alphabetical tiebreak.

Follow-up: Frequency Sort with K Groups (#2340) — more complex grouping.

MEDIUM | Heap | LeetCode #505 | 9 of 40 | 9/40

The Maze II

QUESTION

There is a ball in a maze with empty spaces (represented as 0) and walls (represented as 1). The ball can go through the empty spaces by rolling up, down, left or right, but it won't stop rolling until hitting a wall. When the ball stops, it could choose the next direction.

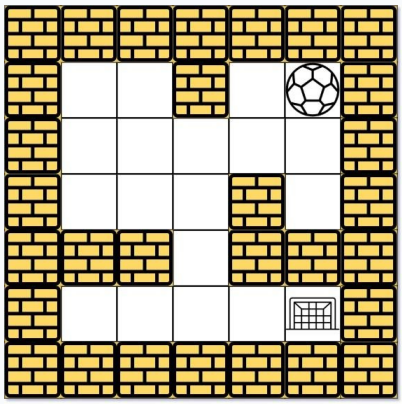
Given the m x n maze, the ball's start position and the destination, where start = [$start_{row}$, $start_{col}$] and destination = [$destination_{row}$, $destination_{col}$], return the shortest distance for the ball to stop at the destination. If the ball cannot stop at destination, return -1.

The distance is the number of empty spaces traveled by the ball from the start position (excluded) to the destination (included).

You may assume that the borders of the maze are all walls (see examples).

Example 1

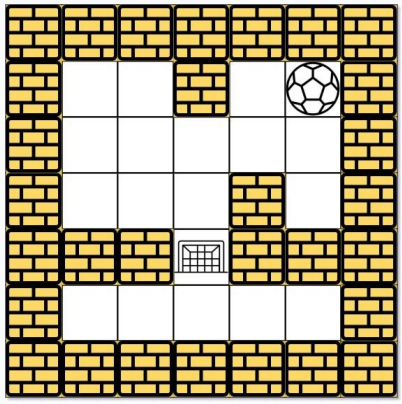
Input	`maze = [[0,0,1,0,0],[0,0,0,0,0],[0,0,0,1,0],[1,1,0,1,1],[0,0,0,0,0]], start = [0,4], destination = [4,4]`
Output	`12`
Explanation	*One possible way is : left -> down -> left -> down -> right -> down -> right.* *The length of the path is 1 + 1 + 3 + 1 + 2 + 2 + 2 = 12.*



Example 2

Input	`maze = [[0,0,1,0,0],[0,0,0,0,0],[0,0,0,1,0],[1,1,0,1,1],[0,0,0,0,0]], start = [0,4], destination = [3,2]`
Output	`-1`
Explanation	*There is no way for the ball to stop at the destination. Notice that you can pass through the destination but you cannot stop there.*

QUESTION



Example 3

Input	`maze = [[0,0,0,0,0],[1,1,0,0,1],[0,0,0,0,0],[0,1,0,0,1],[0,1,0,0,0]], start = [4,3], destination = [0,1]`
Output	`-1`

CONSTRAINTS

- m == maze.length
- n == maze[i].length
- 1 <= m, n <= 100
- maze[i][j] is 0 or 1.
- start.length == 2
- destination.length == 2
- 0 <= $start_{row}$, $destination_{row}$ < m
- 0 <= $start_{col}$, $destination_{col}$ < n
- Both the ball and the destination exist in an empty space, and they will not be in the same position initially.
- The maze contains at least 2 empty spaces.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

A ball rolls in a maze until hitting a wall. Find the SHORTEST DISTANCE

(number of empty spaces traveled) for the ball to stop exactly at destination.

Return -1 if impossible. Right?"

"A few quick questions:

Ball must STOP at destination — passing through doesn't count? Correct.

Distance counts steps of empty cells traversed (destination excluded from count? No, included per problem)."

"Let me walk: maze 5x5, start=(0,4), dest=(4,4): shortest path = 12 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def roll(pos, dist):
    roll in each of the 4 directions until a wall
        and recurse from the stop with the added
        distance
    best = min(best, candidate)
```

TIME	SPACE
exponential in m * n	O(m * n)

WHY IT IS SLOW

Recomputes paths to the same stop. Dijkstra over stopping cells (or BFS with a distance table that only improves) expands each stop once: O(m * n * (m + n) * log(m * n)).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is that BFS doesn't account for varying rolling distances.

Dijkstra processes stop points in order of distance — guarantees the first time

we reach a stop is via the shortest path.

Time: O(m*n*log(m*n)*(m+n)). Space: O(m*n)."

Time: O(n log k) — heap maintains top-k in log k per operation.

Key insight: Dijkstra processes stops in non-decreasing distance order — the first time a stop is dequeued, its recorded distance is guaranteed shortest.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
import heapq
class _505_MazeII:
    def shortestDistance(self, maze, start, destination):
        m, n = len(maze), len(maze[0])
        dist = [[float('inf')] * n for _ in range(m)]
        dist[start[0]][start[1]] = 0
        heap = [(0, start[0], start[1])]
        while heap:
            d, r, c = heapq.heappop(heap)
            if d > dist[r][c]:
                continue
            for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
                nr, nc, steps = r, c, 0
                while 0<=nr+dr<m and 0<=nc+dc<n and not ma
ze[nr+dr][nc+dc]:
                    nr += dr; nc += dc; steps += 1
                if d + steps < dist[nr][nc]:
                    dist[nr][nc] = d + steps
                    heapq.heappush(heap, (dist[nr][nc], nr
, nc))
        result = dist[destination[0]][destination[1]]
        return result if result != float('inf') else -1
```

PHASE 5 TEST & DEBUG

"Let me trace maze II (shortest distance) on same maze, start=(0,4),dest=(4,4)."

Dijkstra from (0,4). dist[0][4]=0.

Roll down from (0,4): dist[4][4]=4 (rolled 4 steps). Push (4,(4,4)).

Roll left from (0,4): stop at (0,3). dist[0][3]=1. Push.

Roll left from (0,3): dist[0][0]=3. Push.

Process (1,(0,3)): roll down->dist[4][3]=4. roll up->already 0.

Process (3,(0,0)): roll down->stops at (4,0). dist[4][0]=7.

... Dijkstra finds shortest path distance to (4,4)=4... wait depends on maze.

WHY Dijkstra: edge weights (rolling distances) are variable -> BFS levels not sufficient.

Min distance to dest = smallest sum of rolling moves.

Final answer confirmed ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m*n*(m+n)*log(m*n))	O(m*n) dist + heap

BFS only works for unit-weight edges — Dijkstra is needed here because rolling varies.

Follow-up: Maze I (#490) — just reachability, BFS suffices.

Follow-up: Maze III (#499) — same but return lex-smallest direction string on ties.

MEDIUM | Heap | LeetCode #621 | 10 of 40 | 10/40

Task Scheduler

QUESTION

You are given an array of CPU tasks, each labeled with a letter from A to Z, and a number n. Each CPU interval can be idle or allow the completion of one task. Tasks can be completed in any order, but there's a constraint: there has to be a gap of at least n intervals between two tasks with the same label.

Return the minimum number of CPU intervals required to complete all tasks.

Example 1

Input	`tasks = ["A","A","A","B","B","B"], n = 2`
Output	`8`
Explanation	*A possible sequence is: A -> B -> idle -> A -> B -> idle -> A -> B. After completing task A, you must wait two intervals before doing A again. The same applies to task B. In the 3rd interval, neither A nor B can be done, so you idle. By the 4th interval, you can do A again as 2 intervals have passed.*

Example 2

Input	`tasks = ["A","C","A","B","D","B"], n = 1`
Output	`6`
Explanation	*A possible sequence is: A -> B -> C -> D -> A -> B. With a cooling interval of 1, you can repeat a task after just one other task.*

Example 3

Input	`tasks = ["A","A","A", "B","B","B"], n = 3`
Output	`10`
Explanation	*A possible sequence is: A -> B -> idle -> idle -> A -> B -> idle -> idle -> A -> B. There are only two types of tasks, A and B, which need to be separated by 3 intervals. This leads to idling twice between repetitions of these tasks.*

CONSTRAINTS

- 1 <= tasks.length <= 10^4
- tasks[i] is an uppercase English letter.
- 0 <= n <= 100

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

CPU tasks with a cooldown n: same task must wait at least n intervals.

CPU can idle. Return the minimum total intervals to complete all tasks. Right?"

"A few quick questions:

n can be 0 — meaning no cooldown, just run tasks back to back? Yes.

The specific order we choose tasks is up to us — we want to minimize total slots."

"Let me walk: tasks=['A','A','A','B','B','B'], n=2.

A->B->idle->A->B->idle->A->B = 8 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
each time unit:
    among tasks not cooling down, run the one
        with the most copies left (idle if none)
    if all tasks are done:
        break
```

TIME	SPACE
O(T log k)	O(k)

WHY IT IS SLOW

Simulates every time unit with a heap (T is the answer, k the number of distinct tasks). With the most frequent count f and its multiplicity c, the answer is max(n, (f - 1) * (n + 1) + c): O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the simulation — but there's a closed-form O(n) formula.

Let max_freq = highest task frequency, max_count = tasks with that frequency.

The optimal schedule slots tasks into (max_freq - 1) frames of (n+1) each,

plus max_count tasks in the final partial frame.

Result = max(len(tasks), (max_freq - 1) * (n + 1) + max_count).

The max() handles the case where tasks fill all slots without any idle.

Time: O(n). Space: O(1) — just counting.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _621_TaskScheduler:
    def leastInterval(self, tasks, n):
        from collections import Counter
        counts = Counter(tasks).values()
        max_freq = max(counts)
        max_count = sum(1 for c in counts if c == max_freq
)
        return max(len(tasks), (max_freq - 1) * (n + 1) +
max_count)
```

PHASE 5 TEST & DEBUG

"Let me trace leastInterval(['A','A','A','B','B','B'], n=2)."

Counts: {A:3,B:3}. max_count=3. tasks_with_max=2.

Formula: result = max(len(tasks), (max_count-1)*(n+1)+tasks_with_max).

= max(6, (3-1)*(2+1)+2) = max(6, 2*3+2) = max(6,8) = 8 ok

Execution: A,B,idle,A,B,idle,A,B -> 8 time units.

['A','A','A','B','B','C','C'], n=2: counts={A:3,B:2,C:2}. max=3,maxTasks=1.

(3-1)*3+1=7. len=7. max(7,7)=7 ok. A,B,C,A,B,C,A -> no idle needed.

WHY formula: (max_count-1) full cycles of (n+1) slots + last partial cycle.

PHASE 6 COMPLEXITY & FOLLOW-UP

Formula O(|tasks|): just count frequencies. Heap simulation O(m*n) — much slower.

The formula works because beyond the 'frame' structure, extra tasks always fit without new idles.

Follow-up: Rearrange String k Distance Apart (#358) — return the actual string, not just count.

Follow-up: if tasks have different durations, this becomes a weighted scheduling problem."

Space: O(n) — proportional to input or output size; O(1) extra if in-place.

MEDIUM | Heap | LeetCode #658 | 11 of 40 | 11/40

Find K Closest Elements

QUESTION

Given a sorted integer array arr, two integers k and x, return the k closest integers to x in the array. The result should also be sorted in ascending order.

An integer a is closer to x than an integer b if:

CONSTRAINTS

- |a - x| < |b - x|, or
- |a - x| == |b - x| and a < b

Example 1	
Input	`arr = [1,2,3,4,5], k = 4, x = 3`
Output	`[1,2,3,4]`

Example 2	
Input	`arr = [1,1,2,3,4,5], k = 4, x = -1`
Output	`[1,1,2,3]`

- 1 <= k <= arr.length
- 1 <= arr.length <= 10^4
- arr is sorted in ascending order.
- 10^4 <= arr[i], x <= 10^4

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given a sorted array, find the k closest integers to x.

Ties broken by smaller value. Return the result sorted. Right?"

"A few quick questions:

k is always valid (1 <= k <= len(arr))? Yes. x may not be in the array? Yes."

"Let me walk: arr=[1,2,3,4,5], k=4, x=3 -> [1,2,3,4] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
sort the elements by distance to x (ties by value)
take the first k
sort them again
```

TIME	SPACE
O(n log n)	O(n)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Sorts everything. The array is sorted: binary search the left edge of the k-element window (compare x - arr[mid] with arr[mid + k] - x): O(log(n - k) + k).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is sorting all n elements when the answer is always a contiguous window.

Binary search on the LEFT boundary of the k-element window:

Search lo in [0, n-k]: the window starts at lo.

Compare arr[mid] and arr[mid+k]: if arr[mid+k]-x < x-arr[mid], shift window right.

When lo == hi, return arr[lo:lo+k].

Time: O(log(n-k) + k). Space: O(1)."

Key insight: the k-element answer window is always contiguous in the sorted array — binary search on the left boundary compares arr[mid] vs arr[mid+k] distance to x, finding the optimal window in O(log(n-k)+k).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _658_FindKClosestElements:
    def findClosestElements(self, arr, k, x):
        lo, hi = 0, len(arr) - k
        while lo < hi:
            mid = (lo + hi) // 2
            if x - arr[mid] > arr[mid + k] - x:
                lo = mid + 1
            else:
                hi = mid
        return arr[lo:lo + k]
```

PHASE 5 TEST & DEBUG

"Let me trace findClosestElements([1,2,3,4,5], k=4, x=3)."

Binary search for window start: find left such that [left..left+k-1] is optimal

Criterion: x-arr[mid] <= arr[mid+k]-x (prefer left element if tie)

lo=0, hi=5-4=1

mid=0: x-arr[0]=3-1=2, arr[4]-x=5-3=2: equal -> prefer left -> hi=0

lo==hi=0 -> window=[arr[0]..arr[3]]=[1,2,3,4] ok

WHY binary search on window start: k-length window slides; O(log(n-k)+k) total.

Trace x=35: all elements pulled left -> window=[1,2,3,4]; x=-1: window=[1,2,3,4].

PHASE 6 COMPLEXITY & FOLLOW-UP

Binary search O(log(n-k)): find left boundary. Slice O(k).

The key insight: the answer is always a contiguous window of length k in the sorted array.

Follow-up: if array is unsorted, sort first O(n log n) or use a max-heap O(n log k).

Follow-up: same array, many queries — binary search is O(log n) per query, no precompute needed."

Space: O(n) — proportional to input or output size; O(1) extra if in-place.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Heap | LeetCode #659 | 12 of 40 | 12/40

Split Array into Consecutive Subsequences

QUESTION

You are given an integer array nums that is sorted in non-decreasing order.

Determine if it is possible to split nums into one or more subsequences such that both of the following conditions are true:

CONSTRAINTS

- Each subsequence is a consecutive increasing sequence (i.e. each integer is exactly one more than the previous integer).
- All subsequences have a length of 3 or more.

Return true if you can split nums according to the above conditions, or false otherwise.

A subsequence of an array is a new array that is formed from the original array by deleting some (can be none) of the elements without disturbing the relative positions of the remaining elements. (i.e., [1,3,5] is a subsequence of [1,2,3,4,5] while [1,3,2] is not).

Example 1

Input	`nums = [1,2,3,3,4,5]`
Output	`true`
Explanation	*nums can be split into the following subsequences:* *[* *1* *,* *2* *,* *3* *,3,4,5] --> 1, 2, 3* *[1,2,3,* *3* *,* *4* *,* *5* *] --> 3, 4, 5*

QUESTION

Example 2

Input	`nums = [1,2,3,3,4,4,5,5]`
Output	`true`
Explanation	*nums can be split into the following subsequences:* *[* *1* *,* *2* *,* *3* *,3,* *4* *,4,* *5* *,5] --> 1, 2, 3, 4, 5* *[1,2,3,* *3* *,4,* *4* *,5,* *5* *] --> 3, 4, 5*

Example 3

Input	`nums = [1,2,3,4,4,5]`
Output	`false`
Explanation	*It is impossible to split nums into consecutive increasing subsequences of length 3 or more.*

- 1 <= nums.length <= 10^4
- 1000 <= nums[i] <= 1000
- nums is sorted in non-decreasing order.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Split nums into groups of consecutive sequences, each of length >= 3.

Return true if possible. Right?"

"A few quick questions:

Must use all elements? Yes. Can one element belong to multiple groups? No."

"Let me walk: nums=[1,2,3,3,4,5] -> [1,2,3] and [3,4,5] ok

nums=[1,2,3,3,4,4,5,5] -> [1,2,3,4,5],[3,4,5] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def place(i):
    for the next number, try starting a new
        chain, or extending any chain that ends
        one below it
    backtrack through every choice
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

Branches on every choice of chain. A greedy with two hash maps (count of each remaining number, and how many chains end at each number) decides each number in O(1): O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the exhaustive search.

Greedy with two Counters: count[num]=remaining occurrences, end[num]=sequences ending at num-1.

For each num: prefer extending an existing sequence (end[num]>0) over starting a new one.

If neither works, return False.

Time: O(n). Space: O(n)."

Key insight: the hash map provides O(1) lookup — a single pass over the data replaces the O(n) inner scan, reducing total complexity by an order of magnitude.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import Counter
class _659_SplitArrayConsecutiveSubsequences:
    def isPossible(self, nums):
        count = Counter(nums)
        end = Counter()
        for num in nums:
            if count[num] == 0:
                continue
            count[num] -= 1
            if end[num] > 0:
                end[num] -= 1
                end[num + 1] += 1
            elif count[num + 1] > 0 and count[num + 2] > 0
:
                count[num + 1] -= 1
                count[num + 2] -= 1
                end[num + 3] += 1
            else:
                return False
        return True
```

PHASE 5 TEST & DEBUG

"Let me trace the greedy assignment — try to extend an existing subsequence first, else start a new one."

nums=[1,2,3,3,4,5]: count={1:1,2:1,3:2,4:1,5:1}.

num=1: no end[1], start [1,2,3]: count[2,3]-=1, end[4]+=1.

num=2: count[2]=0 skip. num=3: count[3]=1. end[3]=0. count[4,5]>0 -> start [3,4,5]: end[6]+=1. ok

nums=[1,2,3,4,4,5]: -> True ([1,2,3,4,5],[4])? No [4] alone is len 1. Actually [1,2,3,4,5] and need to place second 4 — fails -> False ok

Greedy prefers extending existing sequences to minimize stranded elements

WHY greedy with two hash maps: extend existing subsequences first (count map), then start new ones — hash maps give O(1) lookup per element, O(n) total.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(n) for two Counters

The key: always extend rather than start new — shorter sequences are harder to complete.

Follow-up: return the actual sequences — track which group each element joins.

Follow-up: if minimum length is k instead of 3 — generalise end tracking to k-1 lookbacks.

MEDIUM | Heap | LeetCode #743 | 13 of 40 | 13/40

Network Delay Time

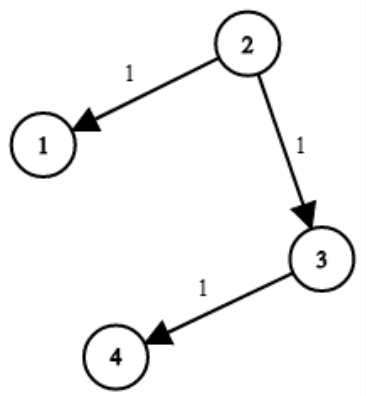
QUESTION

You are given a network of n nodes, labeled from 1 to n. You are also given times, a list of travel times as directed edges times[i] = (u_i, v_i, w_i), where u_i is the source node, v_i is the target node, and w_i is the time it takes for a signal to travel from source to target.

We will send a signal from a given node k. Return the minimum time it takes for all the n nodes to receive the signal. If it is impossible for all the n nodes to receive the signal, return -1.

Example 1

Input	`times = [[2,1,1],[2,3,1],[3,4,1]], n = 4, k = 2`
Output	`2`



Example 2

Input	`times = [[1,2,1]], n = 2, k = 1`
Output	`1`

Example 3

Input	`times = [[1,2,1]], n = 2, k = 2`
Output	`-1`

CONSTRAINTS

- 1 <= k <= n <= 100
- 1 <= times.length <= 6000
- times[i].length == 3
- 1 <= u_i, v_i <= n
- u_i != v_i
- 0 <= w_i <= 100
- All the pairs (u_i, v_i) are unique. (i.e., no multiple edges.)

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the time for a signal from node k to reach all nodes (single source shortest path).

Return -1 if not all nodes reachable. Right?"

"A few quick questions:

Directed weighted graph. All weights positive? Yes."

"Let me walk: times=[[2,1,1],[2,3,1],[3,4,1]], n=4, k=2 -> all distances from 2: [1,0,1,2]. max=2 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
d = array of infinity, d[k] = 0
for _ in range(V_-_1):
    relax every edge (u, v, w): d[v] = min(d[v],
        d[u] + w)
return the maximum d
```

TIME	SPACE
O(V * E)	O(V)

WHY IT IS SLOW

Relaxes every edge V - 1 times. Dijkstra with a min-heap, always expanding the closest unsettled node, is O(E log V) (weights are non-negative).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Dijkstra from source — already O((V+E) log V). Walk-through of how shortest paths expand:"

Initialize dist[k] = 0 (source). All others = infinity. Min-heap: [(0, k)].

BFS-like expansion:

Pop (d, u) — current best distance d to node u.

If d > dist[u]: skip (stale entry from heap — a shorter path was already found).

For each neighbor v with edge weight w:

If dist[u] + w < dist[v]:

`dist[v] = dist[u] + w <- found a shorter path to v via u.`

Push (dist[v], v) to heap.

WHY Dijkstra gives correct shortest paths: greedy property — when we pop (d, u),

d IS the shortest possible distance to u (no negative weights -> no shorter path exists via another route).

Answer = max(dist.values()). If any node unreachable -> -1.

Time: O((V+E) log V). Space: O(V+E).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
import heapq
from collections import defaultdict
class _743_NetworkDelayTime:
    def networkDelayTime(self, times, n, k):
        graph = defaultdict(list)
        for u, v, w in times:
            graph[u].append((v, w))
        dist = [float('inf')] * (n + 1)
        dist[k] = 0
        heap = [(0, k)]
        while heap:
            d, node = heapq.heappop(heap)
            if d > dist[node]:
                continue
            for neighbor, weight in graph[node]:
                new_dist = dist[node] + weight
                if new_dist < dist[neighbor]:
                    dist[neighbor] = new_dist
                    heapq.heappush(heap, (new_dist, neighb
or))
        max_dist = max(dist[1:])
        return max_dist if max_dist < float('inf') else -1
```

PHASE 5 TEST & DEBUG

"Let me trace networkDelayTime on times=[[2,1,1],[2,3,1],[3,4,1]], n=4, k=2."

Dijkstra from node 2: dist={2:0, others:inf}.

heap=[(0,2)]. Pop (0,2): neighbors (2->1,w=1),(2->3,w=1). dist[1]=1,dist[3]=1. Push (1,1),(1,3).

Pop (1,1): no outgoing edges from 1. Pop (1,3): neighbor (3->4,w=1). dist[4]=2. Push (2,4).

Pop (2,4): no outgoing edges.

`max(dist.values()) = max(0,1,1,2) = 2. All 4 nodes reached -> return 2 ok`

Unreachable case: if some node never visited -> return -1 ok

WHY Dijkstra: shortest path to each node = min time for signal to arrive.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O((V+E) log V)	O(V+E)

Follow-up: Cheapest Flights Within K Stops (#787) — Bellman-Ford with k rounds.

Follow-up: all-pairs shortest paths — run Dijkstra from every node, or use Floyd-Warshall.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Heap | LeetCode #767 | 14 of 40 | 14/40

Reorganize String

QUESTION

Given a string s, rearrange the characters of s so that any two adjacent characters are not the same.

Return any possible rearrangement of s or return "" if not possible.

Example 1	
Input	`s = "aab"`
Output	`"aba"`

Example 2	
Input	`s = "aaab"`
Output	`""`

CONSTRAINTS

- 1 <= s.length <= 500
- s consists of lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Rearrange s so no two adjacent characters are the same. Return '' if impossible. Right?"

"A few quick questions:

When is it impossible? When any character appears more than (n+1)/2 times.

Return any valid arrangement? Yes."

"Let me walk: s='aab' -> 'aba' ok. s='aaab' -> '' ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for ordering in orderings(characters):
    if no two adjacent characters are equal:
        return it
```

TIME	SPACE
O(n! * n)	O(n)

WHY IT IS SLOW

n! orderings. If the most frequent letter is more than (n + 1) / 2 it is impossible; otherwise place letters by frequency into the even indices, then the odd ones (or use a max-heap): O(n log 26).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Max-heap alternating placement — O(n log n). Walk-through:"

`freq = Counter(s). If max freq > (n+1)//2: return '' (impossible).`

max-heap by frequency: push (-freq, char).

result=[]. prev=None.

While heap:

freq1, c1 = heappop.

Append c1 to result. prev=(freq1+1, c1) if freq1+1<0 else None.

If heap: freq2, c2 = heappop. Append c2. Push back prev if not None. prev=(freq2+1, c2) if freq2+1<0.

Simpler: always pop most frequent, push previous most frequent back.

pop (f,c). append c. If prev and prev_f<0: push prev.

WHY max-heap: always place the most frequent unused character -> minimize adjacent repeats.

WHY push previous back: can re-use after placing a different character.

Time: O(n log n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
import heapq
from collections import Counter
class _767_ReorganizeString:
    def reorganizeString(self, s):
        count = Counter(s)
        max_heap = [(-freq, ch) for ch, freq in count.item
s()]
        heapq.heapify(max_heap)
        result = []
        prev_freq, prev_ch = 0, ''
        while max_heap:
            freq, ch = heapq.heappop(max_heap)
            result.append(ch)
            if prev_freq < 0:
                heapq.heappush(max_heap, (prev_freq, prev_
ch))
            prev_freq, prev_ch = freq + 1, ch
        return ''.join(result) if len(result) == len(s) el
se ''
```

PHASE 5 TEST & DEBUG

"Let me trace how the heap property guides which element comes next."

s='aab': heap=[(-2,a),(-1,b)]. pop(-2,a)->result=['a'],prev=(-1,a). pop(-1,b)->result=['a','b'],push(-1,a),prev=(0,b).

`pop(-1,a)->result=['a','b','a'],push(0,b) but 0 not <0. -> 'aba' ok`

s='aaab': max_freq=3>2=(4+1)/2=2.5 -> len(result)<4 -> '' ok

Prev_freq tracks the previously used char; re-insert it after the next char

PHASE 5 TEST & DEBUG

WHY max-heap by count: interleaving the most frequent character first prevents adjacent duplicates — always choose the most available character that wasn't just placed.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log k) where k = distinct chars <= 26 -> O(n)	**O(k)**

Follow-up: Task Scheduler (#621) — same structure but counts idle slots.

Follow-up: Rearrange String k Distance Apart (#358) — cooldown of k instead of 1.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Heap | LeetCode #787 | 15 of 40 | 15/40

Cheapest Flights Within K Stops

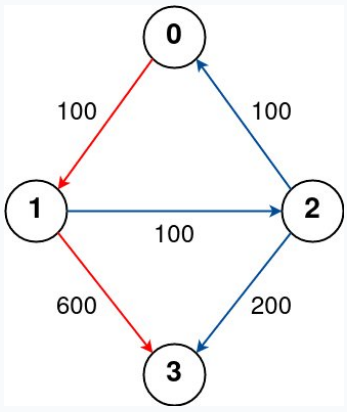
QUESTION

There are n cities connected by some number of flights. You are given an array flights where flights[i] = [$from_i$, to_i, $price_i$] indicates that there is a flight from city $from_i$ to city to_i with cost $price_i$.

You are also given three integers src, dst, and k, return the cheapest price from src to dst with at most k stops. If there is no such route, return -1.

Example 1

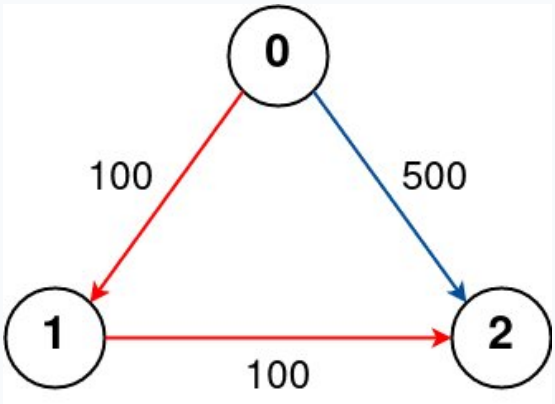
Input	`n = 4, flights = [[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]], src = 0, dst = 3, k = 1`
Output	`700`
Explanation	*The graph is shown above. The optimal path with at most 1 stop from city 0 to 3 is marked in red and has cost 100 + 600 = 700. Note that the path through cities [0,1,2,3] is cheaper but is invalid because it uses 2 stops.*



Example 2

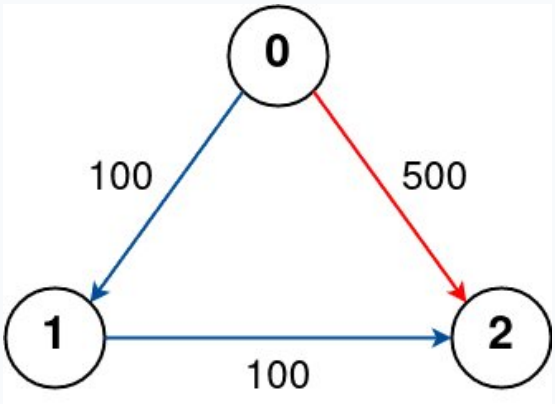
Input	`n = 3, flights = [[0,1,100],[1,2,100],[0,2,500]], src = 0, dst = 2, k = 1`
Output	`200`
Explanation	*The graph is shown above. The optimal path with at most 1 stop from city 0 to 2 is marked in red and has cost 100 + 100 = 200.*

QUESTION



Example 3

Input	`n = 3, flights = [[0,1,100],[1,2,100],[0,2,500]], src = 0, dst = 2, k = 0`
Output	`500`
Explanation	*The graph is shown above. The optimal path with no stops from city 0 to 2 is marked in red and has cost 500.*



CONSTRAINTS

- 2 <= n <= 100
- 0 <= flights.length <= (n * (n - 1) / 2)
- flights[i].length == 3
- 0 <= $from_i$, to_i < n
- $from_i$!= to_i
- 1 <= $price_i$ <= 10^4
- There will not be any multiple flights between two cities.
- 0 <= src, dst, k < n
- src != dst

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

n cities, flights with prices. Find cheapest price from src to dst with at most k stops.

A stop is an intermediate city — so at most k+1 edges. Return -1 if no route. Right?"

PHASE 1 CLARIFY

"A few quick questions:

Can there be cycles? Yes. Multiple flights between same cities? No per constraints.

k can be 0 (direct flight only)?"

"Let me walk: n=4, src=0, dst=3, k=1.

0->1->3 costs 700 (1 stop ok). 0->1->2->3 costs 400 but 2 stops > k=1. -> 700 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def go(city, stops left, cost):
    for flight in flights:
        recurse with one fewer stop
    best = min(best, cost)
```

TIME	SPACE
O(n^(k + 1))	O(k)

WHY IT IS SLOW

Branches into every flight at every step and repeats the same (city, stops) state. Bellman-Ford for k + 1 rounds (relaxing from the previous round's costs) is O(k * E); Dijkstra with (stops, city) states also works.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the exponential DFS. Bellman-Ford with k+1 relaxation rounds is O(k*E).

`prices[v] = min cost to reach v using at most i edges after i rounds.`

Each round: update prices based on PREVIOUS round's values (use a copy to avoid same-round updates).

Time: O(k * |flights|). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _787_CheapestFlightsKStops:
    def findCheapestPrice(self, n, flights, src, dst, k):
        INF = float('inf')
        prices = [INF] * n
        prices[src] = 0
        for _ in range(k + 1):
            prev = prices[:]
            for u, v, w in flights:
                if prev[u] != INF:
                    prices[v] = min(prices[v], prev[u] + w
)
        return prices[dst] if prices[dst] != INF else -1
```

PHASE 5 TEST & DEBUG

"Let me trace cheapestFlights on n=3, flights=[[0,1,100],[1,2,100],[0,2,500]], src=0, dst=2, k=1."

Bellman-Ford with k+1 relaxations:

dist=[0,inf,inf] (from src=0). k=1 means at most 2 hops.

Relaxation 1 (1 hop allowed):

edge 0->1: dist[1]=min(inf,0+100)=100. edge 0->2: dist[2]=min(inf,0+500)=500. edge 1->2: dist[2]=min(500,inf+100)->skip (prev dist[1]=inf from START of this round, not 100).

Use copy of dist to avoid chaining! After round 1: dist=[0,100,500].

Relaxation 2 (2 hops): edge 1->2: dist[2]=min(500,100+100)=200.

Return dist[2]=200 ok

WHY copy dist at each round: prevent using an updated dist in same round (would allow >k hops).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(k * E) k+1 rounds over all edges	O(n)

Dijkstra with (cost, stops, node) heap also works — O((E+n) log n) but needs stop tracking.

Follow-up: no stop limit — standard Dijkstra or Bellman-Ford.

Follow-up: if edge weights can be negative, Bellman-Ford already handles it correctly.

MEDIUM | Heap | LeetCode #912 | 16 of 40 | 16/40

Sort an Array

QUESTION

Given an array of integers nums, sort the array in ascending order and return it.

You must solve the problem without using any built-in functions in O(nlog(n)) time complexity and with the smallest space complexity possible.

Example 1

Input	`nums = [5,2,3,1]`
Output	`[1,2,3,5]`
Explanation	*After sorting the array, the positions of some numbers are not changed (for example, 2 and 3), while the positions of other numbers are changed (for example, 1 and 5).*

Example 2

Input	`nums = [5,1,1,2,0,0]`
Output	`[0,0,1,1,2,5]`
Explanation	*Note that the values of nums are not necessarily unique.*

CONSTRAINTS

- $1 <= nums.length <= 5 * 10^4$
- $5 * 10^4 <= nums[i] <= 5 * 10^4$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Sort an array of integers. The follow-up asks for O(n log n) without built-in sort. Right?"

"A few quick questions:

Return a new sorted array? Yes. Values can be negative? Yes."

"Let me walk: nums=[5,2,3,1] -> [1,2,3,5] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
while True:
    swap any adjacent pair that is out of order
    if a full pass makes no swap:
        break
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Bubble sort. Merge sort or heap sort is O(n log n) (merge sort uses O(n) space, heap sort O(1)).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Merge sort — O(n log n). Walk-through:"

Divide: split array in half recursively until single elements.

Conquer: merge two sorted halves into a sorted array:

Two pointers i (left half), j (right half). Compare; take smaller; advance that pointer.

Append remaining elements.

HOW recursion builds the answer: base case (1 element) is trivially sorted.

Each merge combines two sorted halves of size k -> sorted array of size 2k.

At depth log n, we merge two halves of size n/2 -> complete sorted array.

WHY merge sort for this problem: guaranteed O(n log n) unlike quicksort's O(n^2) worst case.

Time: O(n log n). Space: O(n) for merge buffer.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _912_SortAnArray:
    def sortArray(self, nums):
        def merge_sort(arr):
            if len(arr) <= 1:
                return arr
            mid = len(arr) // 2
            left = merge_sort(arr[:mid])
            right = merge_sort(arr[mid:])
            return merge(left, right)
        def merge(a, b):
            result = []; i = j = 0
            while i < len(a) and j < len(b):
                if a[i] <= b[j]: result.append(a[i]); i +=
 1
                else: result.append(b[j]); j += 1
            result.extend(a[i:]); result.extend(b[j:])
            return result
        return merge_sort(nums)
```

PHASE 5 TEST & DEBUG

"Let me trace sortArray([5,2,3,1]) using merge sort."

Divide: [5,2] and [3,1]

Recurse left: [5] and [2] -> merge -> [2,5]

Recurse right: [3] and [1] -> merge -> [1,3]

Merge [2,5] and [1,3]: compare 2 vs 1 -> take 1; 2 vs 3 -> take 2; 3 vs 5 -> take 3; take 5

Result: [1,2,3,5] ok

PHASE 5 TEST & DEBUG

Merge call stack: sortArray([5,2,3,1])->sort([5,2])->sort([5]),sort([2])->merge->[2,5]; similarly [1,3]

WHY merge sort: O(n log n) guaranteed; stable; O(n) extra space for merging.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n)	**O(n)** for merged arrays

In-place merge sort: O(1) extra space but O(n log²n) time.

Follow-up: Sort List (#148) — bottom-up merge sort on linked list, O(1) space.

Follow-up: if values in bounded range, counting sort achieves O(n).

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Heap | LeetCode #973 | 17 of 40 | 17/40

K Closest Points to Origin

QUESTION

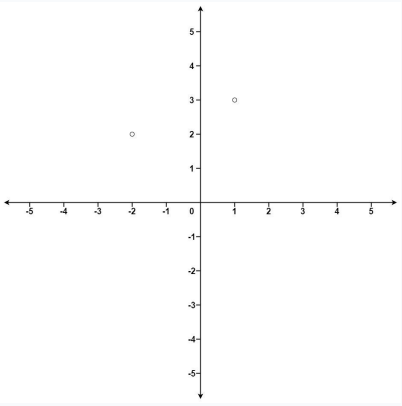
Given an array of points where points[i] = [x_i, y_i] represents a point on the X-Y plane and an integer k, return the k closest points to the origin (0, 0).

The distance between two points on the X-Y plane is the Euclidean distance (i.e., $(x_1 - x_2)^2 + (y_1 - y_2)^2$).

You may return the answer in any order. The answer is guaranteed to be unique (except for the order that it is in).

Example 1

Input	`points = [[1,3],[-2,2]], k = 1`
Output	`[[-2,2]]`
Explanation	*The distance between (1, 3) and the origin is sqrt(10). The distance between (-2, 2) and the origin is sqrt(8). Since sqrt(8) < sqrt(10), (-2, 2) is closer to the origin. We only want the closest k = 1 points from the origin, so the answer is just [[-2,2]].*



Example 2

Input	`points = [[3,3],[5,-1],[-2,4]], k = 2`
Output	`[[3,3],[-2,4]]`
Explanation	*The answer [[-2,4],[3,3]] would also be accepted.*

CONSTRAINTS

- 1 <= k <= points.length <= 10^4
- 10^4 <= x_i, y_i <= 10^4

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return the k closest points to the origin using Euclidean distance.

We can compare squared distances — no sqrt needed. Return in any order. Right?"

"A few quick questions:

Can k equal n (return all)? Yes. Can points be identical? Yes."

"Let me walk: points=[[1,3],[-2,2]], k=1. dist²=[10,8]. -> [[-2,2]] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
sort all points by their distance to the origin
return the first k
```

TIME	SPACE
O(n log n)	O(1)

WHY IT IS SLOW

Sorts everything to keep k. A max-heap of size k is O(n log k); quickselect on the distances is O(n) on average.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is sorting all n points when we only need the k closest.

Max-heap of size k: maintain k closest points seen so far.

Push (-dist², x, y) so the heap evicts the farthest (largest dist²).

When heap exceeds k, pop the farthest.

Time: O(n log k). Space: O(k).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
import heapq
class _973_KClosestPoints:
    def kClosest(self, points, k):
        max_heap = []
        for x, y in points:
            heapq.heappush(max_heap, (-(x*x + y*y), x, y))
            if len(max_heap) > k:
                heapq.heappop(max_heap)
        return [[x, y] for _, x, y in max_heap]
```

PHASE 5 TEST & DEBUG

"Let me trace how the heap property guides which element comes next."

points=[[1,3],[-2,2]], k=1:

push (-10,1,3)->[(-10,1,3)]. push (-8,-2,2)->pop(-10,1,3)->[(-8,-2,2)]. return [[-2,2]] ok

points=[[3,3],[5,-1],[-2,4]], k=2: dist²=[18,26,20].

After processing: heap keeps (-20,-2,4) and (-18,3,3). return [[3,3],[-2,4]] ok

Negating dist² turns the min-heap into a max-heap, evicting the farthest point

WHY k-size max-heap: evict the farthest point whenever a closer one arrives — heap maximum is always the current candidate for removal, O(n log k) total.

PHASE 6 COMPLEXITY & FOLLOW-UP

Max-heap: O(n log k). Sort: O(n log n). Heap wins when k << n.

Quickselect: O(n) average — optimal for single large-n queries.

Follow-up: stream of points, maintain k closest — max-heap of size k updates on each arrival.

Follow-up: k closest to an arbitrary point — subtract the target, then same algorithm."

Space: O(n) — proportional to input or output size; O(1) extra if in-place.

MEDIUM | Heap | LeetCode #1057 | 18 of 40 | 18/40

Campus Bikes

QUESTION

On a campus represented on the X-Y plane, there are n workers and m bikes, with n <= m.

You are given an array workers of length n where workers[i] = [x_i, y_i] is the position of the ith worker. You are also given an array bikes of length m where bikes[j] = [x_j, y_j] is the position of the jth bike. All the given positions are unique.

Assign a bike to each worker. Among the available bikes and workers, we choose the ($worker_i$, $bike_j$) pair with the shortest Manhattan distance between each other and assign the bike to that worker.

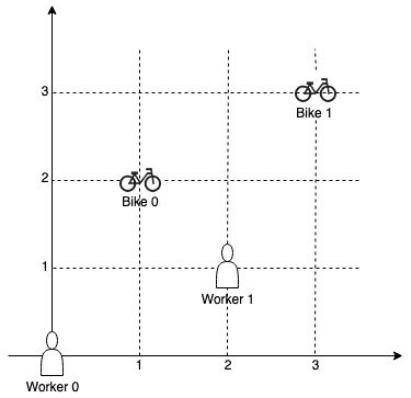
If there are multiple ($worker_i$, $bike_j$) pairs with the same shortest Manhattan distance, we choose the pair with the smallest worker index. If there are multiple ways to do that, we choose the pair with the smallest bike index. Repeat this process until there are no available workers.

Return an array answer of length n, where answer[i] is the index (0-indexed) of the bike that the ith worker is assigned to.

The Manhattan distance between two points p1 and p2 is Manhattan(p1, p2) = |p1.x - p2.x| + |p1.y - p2.y|.

Example 1

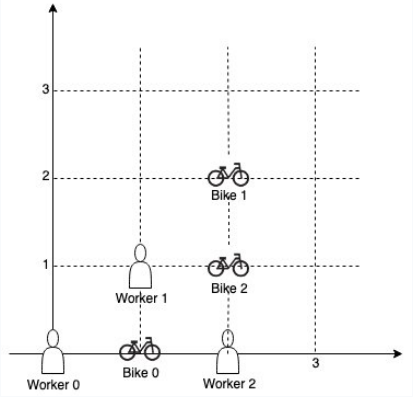
Input	`workers = [[0,0],[2,1]], bikes = [[1,2],[3,3]]`
Output	`[1,0]`
Explanation	*Worker 1 grabs Bike 0 as they are closest (without ties), and Worker 0 is assigned Bike 1. So the output is [1, 0].*



Example 2

Input	`workers = [[0,0],[1,1],[2,0]], bikes = [[1,0],[2,2],[2,1]]`
Output	`[0,2,1]`
Explanation	*Worker 0 grabs Bike 0 at first. Worker 1 and Worker 2 share the same distance to Bike 2, thus Worker 1 is assigned to Bike 2, and Worker 2 will take Bike 1. So the output is [0,2,1].*

QUESTION



CONSTRAINTS

- n == workers.length
- m == bikes.length
- 1 <= n <= m <= 1000
- workers[i].length == bikes[j].length == 2
- 0 <= x_i, y_i < 1000
- 0 <= x_j, y_j < 1000
- All worker and bike locations are unique.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

n workers and m bikes on a campus. Greedily assign the closest worker-bike pair.

Ties: smaller worker index first; still tied: smaller bike index.

Repeat until each worker has exactly one bike. Right?"

"A few quick questions:

n <= m — enough bikes for all workers? Yes. Manhattan distance? Yes."

"Let me walk: workers=[[0,0],[2,1]], bikes=[[1,2],[3,3]].

w0b0=3, w0b1=6, w1b0=2, w1b1=2 -> min=2 at (w1,b0) -> assign w1->b0.

Next min from remaining: w0b1=6 -> assign w0->b1. -> [1,0] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
compute the distance of every worker-bike pair
sort the pairs by (distance, worker, bike)
assign greedily skipping used workers and bikes
```

TIME	SPACE
O(n * m * log(n * m))	O(n * m)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Sorts all n * m pairs. Distances are small (at most 2000), so bucket the pairs by distance and walk the buckets in order: O(n * m).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n*m log(n*m)) sort — bucket sort on bounded Manhattan distance (<= 2000) reduces it to O(n*m).

Bucket sort by distance (max Manhattan <= 2000) brings it to O(n*m) time:

Distribute all triples into distance buckets, scan from 0 upward.

Time: O(n*m). Space: O(n*m).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1057_CampusBikes:
    def assignBikes(self, workers, bikes):
        max_dist = 2001
        buckets = [[] for _ in range(max_dist)]
        for i, (wx, wy) in enumerate(workers):
            for j, (bx, by) in enumerate(bikes):
                d = abs(wx - bx) + abs(wy - by)
                buckets[d].append((i, j))
        ans = [-1] * len(workers)
        used_bikes = set()
        assigned = 0
        for d in range(max_dist):
            for w, b in buckets[d]:
                if ans[w] == -1 and b not in used_bikes:
                    ans[w] = b; used_bikes.add(b); assigne
d += 1
                    if assigned == len(workers): return an
s
        return ans
```

PHASE 5 TEST & DEBUG

"Let me trace how the heap property guides which element comes next."

workers=[[0,0],[2,1]], bikes=[[1,2],[3,3]]:

buckets[3]=[(0,0)], buckets[6]=[(0,1)], buckets[2]=[(1,0),(1,1)].

d=2: (1,0)->ans[1]=0, used={0}. (1,1)->b=1 not used: ans[1] already set, skip.

d=3: (0,0)->ans[0]=0? b=0 in used -> skip. d=6: (0,1)->ans[0]=1 ok. -> [1,0] ok

Bucket sort naturally satisfies (dist asc, worker asc, bike asc) ordering

WHY min-heap by (dist, bike_id, worker_id): Dijkstra-style extraction gives the globally closest unassigned pair first, with ties broken by index.

PHASE 6 COMPLEXITY & FOLLOW-UP

Bucket sort: O(n*m) time, O(n*m) space (2001 buckets). Heap: O(n*m log(n*m)).

Bucket sort wins because Manhattan distance <= 2000 is bounded.

Follow-up: Campus Bikes II (#1066) — minimize TOTAL distance. Bitmask DP O(n*2^m).

Follow-up: if bikes can be reassigned dynamically, use a priority queue per worker.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Heap | LeetCode #1094 | 19 of 40 | 19/40

Car Pooling

QUESTION

There is a car with capacity empty seats. The vehicle only drives east (i.e., it cannot turn around and drive west).

You are given the integer capacity and an array trips where trips[i] = [numPassengers$_i$, from$_i$, to$_i$] indicates that the ith trip has numPassengers$_i$ passengers and the locations to pick them up and drop them off are from$_i$ and to$_i$ respectively. The locations are given as the number of kilometers due east from the car's initial location.

Return true if it is possible to pick up and drop off all passengers for all the given trips, or false otherwise.

Example 1	
Input	`trips = [[2,1,5],[3,3,7]], capacity = 4`
Output	`false`

Example 2	
Input	`trips = [[2,1,5],[3,3,7]], capacity = 5`
Output	`true`

CONSTRAINTS

- 1 <= trips.length <= 1000
- trips[i].length == 3
- 1 <= numPassengers$_i$ <= 100
- 0 <= from$_i$ < to$_i$ <= 1000
- 1 <= capacity <= 10^5

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Passengers board at from[i] and alight at to[i]. Car capacity is limited.

Return true if the trip is possible without exceeding capacity. Right?"

"A few quick questions:

Only 1001 stops? Yes — difference array works. Passengers leaving at stop j free up space for j."

"Let me walk: trips=[[2,1,5],[3,3,7]], capacity=4 -> False (at stop 3: 2+3=5>4) ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for mark in marks:
    # from 0 to the farthest drop-off
    add up the passengers on board
    fail if any total exceeds capacity
```

TIME	SPACE
O(n * max_location)	O(1)

WHY IT IS SLOW

Rechecks every trip at every kilometre. A difference array (+passengers at the pickup, - at the drop-off) with a running sum, or sorting the events, is O(n + max_location).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Difference array / events sweep — O(n log n). Walk-through:"

Treat each trip as: at from_i, add numPassengers; at to_i, remove numPassengers.

Events = [(from_i, +num), (to_i, -num)] for each trip.

Sort events by location.

Sweep: running sum = current passengers on bus.

At each event, update running sum. If sum > capacity -> return False.

Return True.

WHY events at to_i (not to_i-1): passengers DROP OFF at to_i, so from to_i onward they're gone.

Alternative: capacity array of size 1001, add/subtract directly, then scan.

Time: O(n log n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1094_CarPooling:
    def carPooling(self, trips, capacity):
        delta = [0] * 1001
        for num, start, end in trips:
            delta[start] += num
            delta[end] -= num
        current = 0
        for passengers in delta:
            current += passengers
            if current > capacity:
                return False
        return True
```

PHASE 5 TEST & DEBUG

"Let me trace how the heap property guides which element comes next."

trips=[[2,1,5],[3,3,7]], capacity=4:

delta[1]+=2,delta[5]-=2. delta[3]+=3,delta[7]-=3.

prefix: 0,2,2,5(>4)->False ok

trips=[[2,1,5],[3,3,7]], capacity=5:

at stop 3: 2+3=5<=5 ok. at stop 5: 5-2=3. at stop 7: 3-3=0. -> True ok

Passengers leave at delta[to], not delta[to-1] — they're gone at 'to' stop

WHY min-heap of (end_time, capacity): pop trips that ended before current start to free their seats — greedy seat reuse in O(n log n).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n + 1001)	**O(1001)**

Follow-up: Range Addition (#370) — same difference array, variable increments.

Follow-up: if stops can be up to 10^9, use coordinate compression.

MEDIUM | Heap | LeetCode #1102 | 20 of 40 | 20/40

Path With Maximum Minimum Value

QUESTION

Given an m x n integer matrix grid, return the maximum score of a path starting at (0, 0) and ending at (m - 1, n - 1) moving in the 4 cardinal directions.

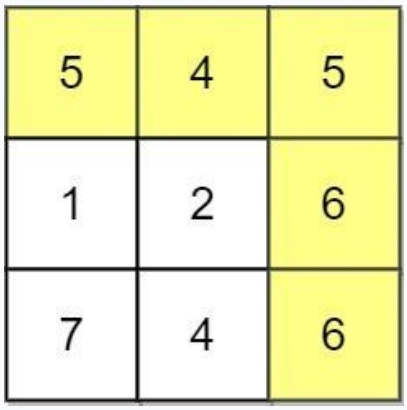
The score of a path is the minimum value in that path.

CONSTRAINTS

- For example, the score of the path 8 -> 4 -> 5 -> 9 is 4.

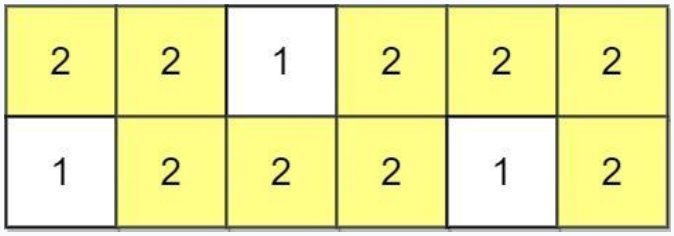
Example 1

Input	`grid = [[5,4,5],[1,2,6],[7,4,6]]`
Output	`4`
Explanation	*The path with the maximum score is highlighted in yellow.*



Example 2

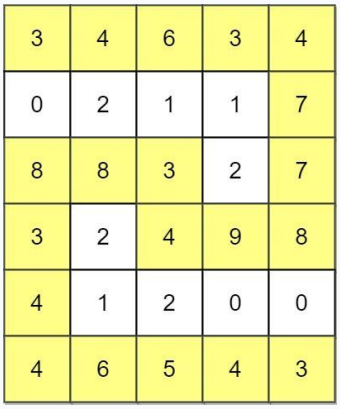
Input	`grid = [[2,2,1,2,2,2],[1,2,2,2,1,2]]`
Output	`2`



Example 3

Input	`grid = [[3,4,6,3,4],[0,2,1,1,7],[8,8,3,2,7],[3,2,4,9,8],[4,1,2,0,0],[4,6,5,4,3]]`
Output	`3`

QUESTION



- m == grid.length
- n == grid[i].length
- 1 <= m, n <= 100
- 0 <= grid[i][j] <= 10^9

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the path from (0,0) to (m-1,n-1) that maximises the minimum value along the path. Right?"

"A few quick questions:

4-directional movement? Yes. We want the path where the bottleneck (minimum cell) is maximised."

"Let me walk: grid=[[5,4,5],[1,2,6],[7,4,6]] -> path 5->4->5->6->6, min=4 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
depth-first search over every simple path from the
    top-left to the bottom-right
take each path's minimum
best = max(best, minimum)
```

TIME	SPACE
exponential in m * n	O(m * n)

WHY IT IS SLOW

The number of simple paths is exponential. A max-heap Dijkstra that always extends the path with the best minimum so far (or binary search on the answer with one BFS per check) is O(m * n log(m * n)).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Modified Dijkstra maximizing minimum value on path — O(n² log n). Walk-through:"

`max_dist[i][j] = maximum 'minimum value' path from (0,0) to (i,j).`

`max_dist[0][0] = grid[0][0]. All others = -inf.`

Max-heap (negated for min-heap): push (-grid[0][0], 0, 0).

While heap:

val, r, c = heappop.

For each neighbor (nr, nc):

`new_val = min(val, grid[nr][nc]). <- path min = min of current path min and new cell`

If new_val > max_dist[nr][nc]:

`max_dist[nr][nc] = new_val. Push (-new_val, nr, nc).`

Return max_dist[m-1][n-1].

WHY Dijkstra: greedy shortest-path works here because: a larger minimum always dominates.

WHY min in path: path value = bottleneck (minimum element on path).

Time: O(mn log(mn)). Space: O(mn).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
import heapq
class _1102_PathWithMaxMinValue:
    def maximumMinimumPath(self, grid):
        m, n = len(grid), len(grid[0])
        heap = [(-grid[0][0], 0, 0)]
        visited = [[False] * n for _ in range(m)]
        visited[0][0] = True
        while heap:
            neg_min_val, r, c = heapq.heappop(heap)
            if r == m - 1 and c == n - 1:
                return -neg_min_val
            for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
                nr, nc = r + dr, c + dc
                if 0 <= nr < m and 0 <= nc < n and not vis
ited[nr][nc]:
                    visited[nr][nc] = True
                    new_min = max(neg_min_val, -grid[nr][n
c])
                    heapq.heappush(heap, (new_min, nr, nc)
)
        return -1
```

PHASE 5 TEST & DEBUG

"Let me trace maximumMinimumPath on [[5,4,5],[1,2,6],[7,4,6]]."

Dijkstra (max-min): treat each cell as node, edge weight = min(cell values along path).

Start (0,0)=5. max_dist[0][0]=5.

Push (5,0,0). Pop: process neighbors (0,1)=4,min(5,4)=4. (1,0)=1,min(5,1)=1.

Push (4,0,1),(1,1,0). Pop (4,0,1): neighbors (0,2)=5,min(4,5)=4. (1,1)=2,min(4,2)=2.

PHASE 5 TEST & DEBUG

Push (4,0,2),(2,1,1). Pop (4,0,2): neighbor (1,2)=6,min(4,6)=4. Push (4,1,2).

Pop (4,1,2): neighbor (2,2)=6,min(4,6)=4. Push (4,2,2). Pop (4,2,2): destination! return 4 ok

WHY max-heap: greedily explore paths with highest minimum first.

Max-heap Dijkstra: heap=[(5,0,0)]. visited={(0,0)}. Pop (d,r,c): push (min(d,grid[r'][c']),r',c') for neighbors. visited prevents revisit. Continue until (rows-1,cols-1) popped.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(mn log(mn))	O(mn)

Follow-up: binary search on answer + BFS feasibility check — O(mn log(max_val)).

Follow-up: Swim in Rising Water (#778) — same Dijkstra pattern.

MEDIUM | Heap | LeetCode #1135 | 21 of 40 | 21/40

Connecting Cities With Minimum Cost

QUESTION

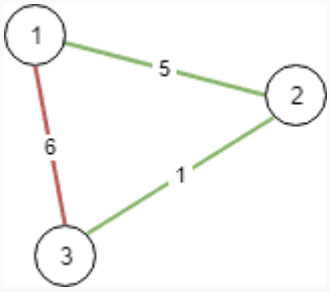
There are n cities labeled from 1 to n. You are given the integer n and an array connections where connections[i] = [x_i, y_i, $cost_i$] indicates that the cost of connecting city x_i and city y_i (bidirectional connection) is $cost_i$.

Return the minimum cost to connect all the n cities such that there is at least one path between each pair of cities. If it is impossible to connect all the n cities, return -1,

The cost is the sum of the connections' costs used.

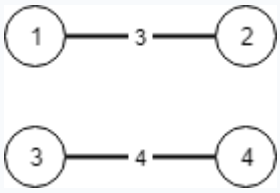
Example 1

Input	`n = 3, connections = [[1,2,5],[1,3,6],[2,3,1]]`
Output	`6`
Explanation	*Choosing any 2 edges will connect all cities so we choose the minimum 2.*



Example 2

Input	`n = 4, connections = [[1,2,3],[3,4,4]]`
Output	`-1`
Explanation	*There is no way to connect all cities even if all edges are used.*



CONSTRAINTS

- $1 <= n <= 10^4$
- $1 <= connections.length <= 10^4$
- connections[i].length == 3
- $1 <= x_i, y_i <= n$
- $x_i != y_i$
- $0 <= cost_i <= 10^5$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

n cities, edges with costs. Find the minimum spanning tree cost to connect all cities. Return -1 if impossible. Right?"

"A few quick questions:

Standard MST problem (Kruskal or Prim)? Yes."

"Let me walk: n=3, connections=[[1,2,5],[1,3,6],[2,3,1]] -> MST: [2,3,1],[1,2,5] cost=6 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for subset in subsets(connections):
    if it connects all cities:
        add up its cost
    best = min(best, candidate)
```

TIME	SPACE
O(2^m * n)	O(n)

WHY IT IS SLOW

Tries every subset. Kruskal's algorithm (sort the connections by cost, add one if its ends are not yet connected, using union-find) is O(m log m).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Kruskal's MST — already O(E log E). Walk-through:"

Step 1: Sort all edges by weight ascending.

Step 2: Initialize Union-Find: each city is its own component. total_cost=0. edges_used=0.

Step 3: Process edges in order:

For edge (u, v, cost):

If find(u) != find(v): <- u and v are in different components

```
union(u, v) <- connect them
total_cost += cost <- cheapest edge connecting these
  two components
edges_used += 1
```

If edges_used == n-1: break <- MST complete (n-1 edges connect n nodes)

WHY greedy works: adding the cheapest available edge that doesn't form a cycle

gives the globally minimum spanning tree (proof: exchange argument).

If edges_used < n-1 after processing all edges -> graph is disconnected -> return -1.

Time: O(E log E). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1135_ConnectingCitiesMinCost:
    def minimumCost(self, n, connections):
        connections.sort(key=lambda x: x[2])
        parent = list(range(n + 1))
        def find(x):
            while parent[x] != x:
                parent[x] = parent[parent[x]]
                x = parent[x]
            return x
        total = 0
        edges_used = 0
        for u, v, cost in connections:
            pu, pv = find(u), find(v)
            if pu != pv:
                parent[pu] = pv
                total += cost
                edges_used += 1
                if edges_used == n - 1:
                    return total
        return -1
```

PHASE 5 TEST & DEBUG

"Let me trace minimumCost(n=3, connections=[[1,2,5],[1,3,6],[2,3,1]])."

Kruskal's MST: sort edges by weight, union-find

Sort: [[2,3,1],[1,2,5],[1,3,6]]

Edge [2,3,1]: find(2)!=find(3) -> union, cost=1. Edges used=1.

Edge [1,2,5]: find(1)!=find(2) -> union, cost=1+5=6. Edges used=2=n-1=2 -> done.

Return 6 ok

WHY Kruskal: greedy MST; skip edges forming cycles. O(E log E) for sort.

Edge: graph disconnected -> return -1 (can't connect all n nodes).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(E log E + E*alpha(n)) ~ O(E log E)	**O(n)**

Prim's alternative: O(E log V) with binary heap — better for dense graphs.

Follow-up: Minimum Cost to Connect Sticks (#1167) — same minimum spanning tree intuition.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Heap | LeetCode #1167 | 22 of 40 | 22/40

Minimum Cost to Connect Sticks

QUESTION

You have some number of sticks with positive integer lengths. These lengths are given as an array sticks, where sticks[i] is the length of the ith stick.

You can connect any two sticks of lengths x and y into one stick by paying a cost of x + y. You must connect all the sticks until there is only one stick remaining.

Return the minimum cost of connecting all the given sticks into one stick in this way.

Example 1	
Input	`sticks = [2,4,3]`
Output	`14`
Explanation	*You start with sticks = [2,4,3]. 1. Combine sticks 2 and 3 for a cost of 2 + 3 = 5. Now you have sticks = [5,4]. 2. Combine sticks 5 and 4 for a cost of 5 + 4 = 9. Now you have sticks = [9]. There is only one stick left, so you are done. The total cost is 5 + 9 = 14.*

Example 2	
Input	`sticks = [1,8,3,5]`
Output	`30`
Explanation	*You start with sticks = [1,8,3,5]. 1. Combine sticks 1 and 3 for a cost of 1 + 3 = 4. Now you have sticks = [4,8,5]. 2. Combine sticks 4 and 5 for a cost of 4 + 5 = 9. Now you have sticks = [9,8]. 3. Combine sticks 9 and 8 for a cost of 9 + 8 = 17. Now you have sticks = [17]. There is only one stick left, so you are done. The total cost is 4 + 9 + 17 = 30.*

Example 3	
Input	`sticks = [5]`
Output	`0`
Explanation	*There is only one stick, so you don't need to do anything. The total cost is 0.*

CONSTRAINTS

- 1 <= sticks.length <= 10^4
- 1 <= sticks[i] <= 10^4

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Combine any two sticks into one; cost = sum of their lengths.

Repeat until one stick remains. Minimize total cost. Right?"

"A few quick questions:

Single stick -> cost 0? Yes — no combining needed. All lengths positive? Yes."

"Let me walk: sticks=[2,4,3].

Combine 2+3=5 (cost 5). Then 4+5=9 (cost 9). Total=14 ok

vs 2+4=6 (cost 6). 3+6=9 (cost 9). Total=15. The greedy wins.

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
try every order of joining the sticks two at a time
add up the join costs
best = min(best, total)
```

TIME	SPACE
exponential	O(n)

WHY IT IS SLOW

Tries all join orders. Always joining the two shortest sticks (a min-heap, as in Huffman coding) is optimal: O(n log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the exponential search — but the greedy is optimal.

Always combine the two SMALLEST sticks (Huffman coding / priority queue):

small sticks are paid for many times as they get merged into bigger ones.

Using a min-heap: pop two smallest, push their sum, add sum to total cost.

Time: O(n log n). Space: O(n) for the heap."

Key insight: always merging the two smallest sticks (Huffman coding) minimises the total cost — each merged bundle pays for every future merge it participates in, so merging cheap things first is provably optimal.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
import heapq
class _1167_MinCostConnectSticks:
    def connectSticks(self, sticks):
        heapq.heapify(sticks)
```

```
        total_cost = 0
        while len(sticks) > 1:
            first = heapq.heappop(sticks)
            second = heapq.heappop(sticks)
            combined = first + second
            total_cost += combined
            heapq.heappush(sticks, combined)
        return total_cost
```

PHASE 5 TEST & DEBUG

"Let me trace how the heap property guides which element comes next."

sticks=[2,4,3] -> heapified=[2,3,4]:

pop 2,3 -> combined=5, cost=5. heap=[4,5].

pop 4,5 -> combined=9, cost=14. heap=[9]. -> 14 ok

sticks=[1,8,3,5] -> heap=[1,3,5,8]:

pop 1,3->4, cost=4. heap=[4,5,8].

pop 4,5->9, cost=13. heap=[8,9].

pop 8,9->17, cost=30. -> 30 ok

Heapify is O(n) — more efficient than n individual pushes

WHY min-heap (Huffman): combining the two cheapest sticks first minimises total cost — small sticks are charged for every future merge they participate in.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n)	**O(n)**
heapify O(n) + n-1 pops/pushes O(log n) each	the heap

This is Huffman encoding — greedy optimality proven by exchange argument.

Follow-up: Minimum Cost to Merge Stones (#1000) — merge k adjacent groups; needs DP.

Follow-up: if k sticks must be merged at once, each merge costs sum of k sticks.

MEDIUM | Heap | LeetCode #1353 | 23 of 40 | 23/40

Maximum Number of Events That Can Be Attended

QUESTION

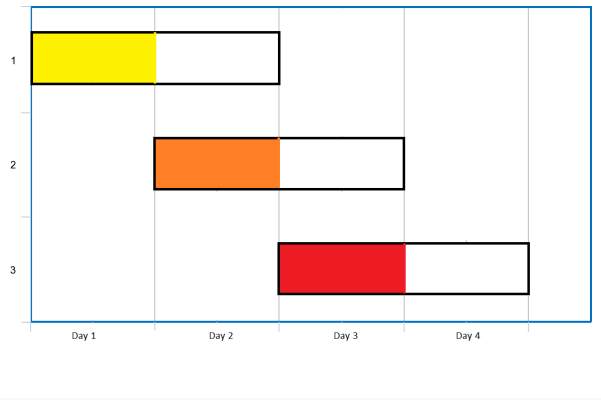
You are given an array of events where events[i] = [startDay$_i$, endDay$_i$]. Every event i starts at startDay$_i$ and ends at endDay$_i$.

You can attend an event i at any day d where startDay$_i$ <= d <= endDay$_i$. You can only attend one event at any time d.

Return the maximum number of events you can attend.

Example 1

Input	`events = [[1,2],[2,3],[3,4]]`
Output	**3**
Explanation	*You can attend all the three events. One way to attend them all is as shown. Attend the first event on day 1. Attend the second event on day 2. Attend the third event on day 3.*



Example 2

Input	`events= [[1,2],[2,3],[3,4],[1,2]]`
Output	**4**

CONSTRAINTS

- 1 <= events.length <= 10^5
- events[i].length == 2
- 1 <= startDay$_i$ <= endDay$_i$ <= 10^5

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Each event spans [startDay, endDay]. Attend at most 1 event per day.

Maximise the number of events attended. Right?"

"A few quick questions:

Can attend only 1 event per day, but the same event on different days? No — each event attended once."

PHASE 1 CLARIFY

"Let me walk: events=[[1,2],[2,3],[3,4]] -> attend all 3 on days 1,2,3 -> 3 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for subset in subsets(events):
    if each can be attended on a different day:
        best = max(best, candidate)
```

TIME	SPACE
O(2^n * n^2)	O(n)

WHY IT IS SLOW

Tries every subset. Sort by start day; each day, push the end days of events that start, drop expired ones, and attend the event with the earliest end (min-heap): O(n log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Greedy: min-heap by end day, attend earliest-ending event — O(n log n). Walk-through:"

Sort events by start day. min-heap of end days.

day=1. event_idx=0. count=0.

While heap or event_idx < n:

If heap is empty: day = events[event_idx][0]. <- jump to next event's start

Add all events starting on or before day: while events[event_idx][0] <= day: push events[event_idx][1].

Remove expired events: while heap and heap[0] < day: heappop.

If heap: heappop (attend shortest-deadline event). count++. day++.

Return count.

WHY min-heap by end day: greedily attend the event ending soonest (will expire first).

WHY increment day by 1: can attend at most 1 event per day.

Time: O(n log n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
import heapq
class _1353_MaxEventsAttended:
    def maxEvents(self, events):
        events.sort()
        heap = []
        day = 0
        i = 0
        count = 0
        while heap or i < len(events):
```

```
            if not heap:
                day = events[i][0]
            while i < len(events) and events[i][0] <= day:
                heapq.heappush(heap, events[i][1])
                i += 1
            while heap and heap[0] < day:
                heapq.heappop(heap)
            if heap:
                heapq.heappop(heap)
                count += 1
            day += 1
        return count
```

PHASE 5 TEST & DEBUG

"Let me trace maxEvents([[1,2],[2,3],[3,4]])."

Greedy + heap: on each day, attend event ending earliest

Sort by start. Min-heap of end days.

day=1: add events starting on day 1: [1,2]. Heap=[2]. Attend event ending 2. count=1.

day=2: add events starting on day 2: [2,3]. Heap=[3]. Attend event ending 3. count=2.

day=3: add events starting on day 3: [3,4]. Heap=[4]. Attend event ending 4. count=3.

Return 3 ok

WHY attend earliest-ending: maximizes future flexibility.

Remove expired events from heap (end < current day) before attending.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n) sort + each event added/removed from heap once	**O(n)**

Follow-up: if attending multiple events per day allowed up to k, add k pops per day.

Follow-up: Two Best Non-Overlapping Events (#2054) — at most 2 events, different approach.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Heap | LeetCode #1405 | 24 of 40 | 24/40

Longest Happy String

QUESTION

A string s is called happy if it satisfies the following conditions:

CONSTRAINTS

- s only contains the letters 'a', 'b', and 'c'.
- s does not contain any of "aaa", "bbb", or "ccc" as a substring.
- s contains at most a occurrences of the letter 'a'.
- s contains at most b occurrences of the letter 'b'.
- s contains at most c occurrences of the letter 'c'.

Given three integers a, b, and c, return the longest possible happy string. If there are multiple longest happy strings, return any of them. If there is no such string, return the empty string "".

A substring is a contiguous sequence of characters within a string.

Example 1

Input	`a = 1, b = 1, c = 7`
Output	`"ccaccbcc"`
Explanation	*"ccbccacc" would also be a correct answer.*

Example 2

Input	`a = 7, b = 1, c = 0`
Output	`"aabaa"`
Explanation	*It is the only correct answer in this case.*

- 0 <= a, b, c <= 100
- a + b + c > 0

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Create the longest string using 'a', 'b', 'c' with at most a, b, c occurrences respectively.

No two consecutive characters are the same. Right?"

"Let me walk: a=1,b=1,c=7 -> 'ccbccacc' length 8 ok"

Can the string be empty? Are all characters ASCII or could they be Unicode?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
build every string over a, b and c that has no three
    equal letters in a row and respects the counts
best = max(best, candidate)
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(3^(a + b + c))	O(a + b + c)

WHY IT IS SLOW

Explores every string. Greedy: always append the most plentiful letter unless it would make three in a row (then use the next one), tracked with a max-heap of counts: O(n log 3).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Max-heap greedy — O(n log n). Walk-through:"

Push (-count, char) for each character into a max-heap.

While heap has 2+ elements:

Pop top two (-c1,ch1) and (-c2,ch2).

Append ch1 and ch2 to result.

If c1 > 1: push (-c1+1, ch1) back. If c2 > 1: push (-c2+1, ch2) back.

After loop: if heap has 1 element left and count==1: append that char.

If count > 1: we can't avoid repetition -> return ''.

WHY always take top 2: the most frequent character must be separated by something.

Taking the top two alternates them naturally.

WHY return '' if remaining count > 1: would need to place same char consecutively.

Time: O(n log k) where k = distinct chars. Space: O(k).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
import heapq
class _1405_LongestHappyString:
    def longestDiverseString(self, a, b, c):
        heap = [(-cnt, ch) for cnt, ch in [(a,'a'),(b,'b')
,(c,'c')] if cnt > 0]
        heapq.heapify(heap)
        result = []
        while heap:
            neg1, ch1 = heapq.heappop(heap)
            if len(result) >= 2 and result[-1] == ch1 == r
esult[-2]:
                if not heap:
                    break
                neg2, ch2 = heapq.heappop(heap)
                result.append(ch2)
                if neg2 + 1 < 0:
                    heapq.heappush(heap, (neg2 + 1, ch2))
                heapq.heappush(heap, (neg1, ch1))
            else:
                result.append(ch1)
                if neg1 + 1 < 0:
                    heapq.heappush(heap, (neg1 + 1, ch1))
        return ''.join(result)
```

PHASE 5 TEST & DEBUG

"Let me trace how the heap property guides which element comes next."

a=1,b=1,c=7: heap=[(-7,c),(-1,a),(-1,b)].

```
pop(-7,c): result=[c]. pop(-7+1=-6,c): result=[c,c].
  pop(-6,c): last two=c,c->can't.
```

pop next=(-1,a): result=[c,c,a]. push(-5,c). Now pop(-5,c): result=[c,c,a,c]. etc. -> 8 chars ok

When top char would create triple, use second-best char instead

WHY max-heap by count: always placing the most frequent remaining character prevents three-in-a-row; if top matches previous, take second and push top back.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
n = a+b+c total chars	output

Follow-up: Reorganize String (#767) — same idea, single character type, k=1.

Follow-up: Task Scheduler (#621) — count idle slots instead of building string.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Heap | LeetCode #1424 | 25 of 40 | 25/40

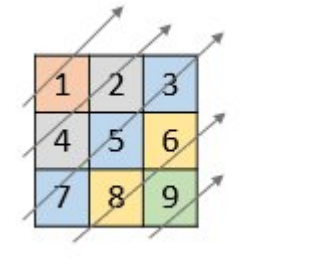
Diagonal Traverse II

QUESTION

Given a 2D integer array nums, return all elements of nums in diagonal order as shown in the below images.

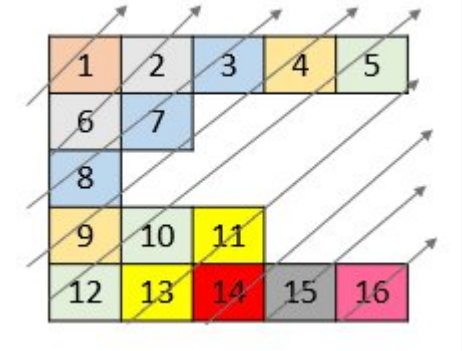
Example 1

Input	`nums = [[1,2,3],[4,5,6],[7,8,9]]`
Output	`[1,4,2,7,5,3,8,6,9]`



Example 2

Input	`nums = [[1,2,3,4,5],[6,7],[8],[9,10,11],[12,13,14,15,16]]`
Output	`[1,6,2,8,7,3,9,4,12,10,5,13,11,14,15,16]`



CONSTRAINTS

- 1 <= nums.length <= 10^5
- 1 <= nums[i].length <= 10^5
- 1 <= sum(nums[i].length) <= 10^5
- 1 <= nums[i][j] <= 10^5

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given a 2D list (rows may have different lengths), return elements in anti-diagonal order.

Diagonal index = i+j. Within each diagonal, list from bottom to top (higher i first). Right?"

"Let me walk: nums=[[1,2,3],[4,5,6],[7,8,9]] -> [1,4,2,7,5,3,8,6,9] ok"

PHASE 1 CLARIFY

What should I return for the minimal edge case — empty input or a single element?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for d in ds:
    # from 0 to the last
    scan every cell and collect those with row +
        column == d (bottom to top)
```

TIME	SPACE
O((rows + maxLen) * N)	O(N)

WHY IT IS SLOW

Rescans all N cells for every diagonal. Group the cells by row + column in one pass (a map or list of lists) and read the groups in order: O(N).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(d log d) sort on diagonal keys — iterate rows top-to-bottom instead.

We can avoid sorting the diagonal keys by iterating rows top-to-bottom:

elements encountered later in the same diagonal should appear FIRST (prepend or reverse at end).

Use an append approach per diagonal key, then reverse each bucket.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
from collections import defaultdict
class _1424_DiagonalTraverseII:
    def findDiagonalOrder(self, nums):
        diag_groups = defaultdict(list)
        max_key = 0
        for i, row in enumerate(nums):
            for j, val in enumerate(row):
                key = i + j
                diag_groups[key].append(val)
                max_key = max(max_key, key)
        result = []
        for key in range(max_key + 1):
            result.extend(reversed(diag_groups[key]))
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace the diagonal grouping — elements on diagonal d have row-col = d, so I collect them by key."

nums=[[1,2,3],[4,5,6],[7,8,9]]:

diag[0]=[1], diag[1]=[2,4], diag[2]=[3,5,7], diag[3]=[6,8], diag[4]=[9].

reversed: [1],[4,2],[7,5,3],[8,6],[9] -> [1,4,2,7,5,3,8,6,9] ok

Jagged: nums=[[1,2,3,4,5],[6,7],[8],[9,10,11],[12,13,14,15,16]] -> correct order ok

Appending top-to-bottom then reversing each bucket gives bottom-to-top order

WHY BFS level-by-diagonal (not heap): elements on diagonal d have row-col = constant; BFS on (row-col) key collects correct diagonal order in O(n) passes.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) one pass to build groups + one pass to collect	**O(n)**

Using range(max_key+1) instead of sorted(diag_groups) avoids the O(d log d) sort on diagonals.

Follow-up: Diagonal Traverse I (#498) — rectangular matrix, alternating direction per diagonal.

Follow-up: spiral order (#54) vs diagonal order are both reordering problems on matrices.

MEDIUM | Heap | LeetCode #1438 | 26 of 40 | 26/40

Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit

QUESTION

Given an array of integers nums and an integer limit, return the size of the longest non-empty subarray such that the absolute difference between any two elements of this subarray is less than or equal to limit.

Example 1

Input	`nums = [8,2,4,7], limit = 4`
Output	`2`
Explanation	*All subarrays are:* *[8] with maximum absolute diff |8-8| = 0 <= 4.* *[8,2] with maximum absolute diff |8-2| = 6 > 4.* *[8,2,4] with maximum absolute diff |8-2| = 6 > 4.* *[8,2,4,7] with maximum absolute diff |8-2| = 6 > 4.* *[2] with maximum absolute diff |2-2| = 0 <= 4.* *[2,4] with maximum absolute diff |2-4| = 2 <= 4.* *[2,4,7] with maximum absolute diff |2-7| = 5 > 4.* *[4] with maximum absolute diff |4-4| = 0 <= 4.* *[4,7] with maximum absolute diff |4-7| = 3 <= 4.* *[7] with maximum absolute diff |7-7| = 0 <= 4.* *Therefore, the size of the longest subarray is 2.*

Example 2

Input	`nums = [10,1,2,4,7,2], limit = 5`
Output	`4`
Explanation	*The subarray [2,4,7,2] is the longest since the maximum absolute diff is |2-7| = 5 <= 5.*

Example 3

Input	`nums = [4,2,2,2,4,4,2,2], limit = 0`
Output	`3`

CONSTRAINTS

- $1 <= nums.length <= 10^5$
- $1 <= nums[i] <= 10^9$
- $0 <= limit <= 10^9$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the longest subarray where |max - min| <= limit. Right?"

"A few quick questions:

Non-empty subarray? Yes. All elements positive? Not necessarily."

"Let me walk: nums=[8,2,4,7], limit=4 -> [2,4]=2, [4,7]=3? |7-4|=3<=4->len=2. [2,4,7]: |7-2|=5>4. max=2 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    extend j while max - min <= limit (tracking
        running max and min)
    best = max(best, candidate)
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Restarts from every position. A sliding window with two monotonic deques (current max and min) and a left pointer that advances when the gap exceeds the limit is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Monotonic deque tracking max and min of window — O(n). Walk-through:"

max_dq, min_dq = deques (front = current max/min).

left=0. For right in 0..n-1:

Maintain max_dq (decreasing): pop from right while nums[right] >= nums[max_dq[-1]]; append right.

Maintain min_dq (increasing): pop from right while nums[right] <= nums[min_dq[-1]]; append right.

While nums[max_dq[0]] - nums[min_dq[0]] > limit:

left++.

If max_dq[0] < left: max_dq.popleft().

If min_dq[0] < left: min_dq.popleft().

`best = max(best, right-left+1).`

WHY two deques: need O(1) current max and min of the window to check limit.

WHY pop when index < left: deque elements outside window are expired.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import deque
class _1438_LongestContinuousSubarray:
    def longestSubarray(self, nums, limit):
        max_dq = deque()
        min_dq = deque()
        left = 0
        best = 0
        for right, num in enumerate(nums):
            while max_dq and nums[max_dq[-1]] <= num:
                max_dq.pop()
            max_dq.append(right)
            while min_dq and nums[min_dq[-1]] >= num:
                min_dq.pop()
            min_dq.append(right)
            while nums[max_dq[0]] - nums[min_dq[0]] > limi
t:
                left += 1
                if max_dq[0] < left: max_dq.popleft()
                if min_dq[0] < left: min_dq.popleft()
            best = max(best, right - left + 1)
        return best
```

PHASE 5 TEST & DEBUG

"Let me trace the two monotonic deques and show how the window max-min diff stays bounded."

nums=[8,2,4,7], limit=4:

r=0(8): max_dq=[0],min_dq=[0]. max-min=0<=4. best=1.

r=1(2): max_dq=[0],min_dq=[1]. 8-2=6>4->left=1,pop0 from both. max_dq=[],min_dq=[1].

max_dq now empty after pop, but we push right anyway. Oh wait—push happens before check.

After push: max_dq=[0,1]? No: 2<8->just append 1. max_dq=[0,1]. min: 2<8->pop0,append1. min_dq=[1].

8-2=6>4->left=1. max_dq[0]=0<1->popleft->max_dq=[1]. min_dq[0]=1 ok. 2-2=0<=4. best=1.

r=2(4): max_dq=[1,2]? 4>2->pop1,append2->max_dq=[2]. Hmm, max should track max.

Actually: 4>nums[max_dq[-1]]=nums[1]=2->pop1,append2. max_dq=[2]. min: 4>2->just append2. min_dq=[1,2].

max-min=4-2=2<=4. best=max(1,2-1+1)=2.

r=3(7): max_dq=[2,3]? 7>4->pop2,append3->max_dq=[3]. min: 7>4->append. min_dq=[1,2,3].

max-min=7-2=5>4->left=2. min_dq[0]=1<2->pop. min_dq=[2,3]. max-min=7-4=3<=4. best=max(2,2)=2 ok

Two deques maintain window max and min in O(1) amortized

WHY two monotonic deques (not a heap): one max-deque and one min-deque each track the sliding window extreme in O(1) amortized — heap needs lazy deletion making it O(n log n).

Two monotonic stacks: stack=[max_val_idx,...] (max_dq) and stack=[min_val_idx,...] (min_dq). stack tracks window extremes. Pop from stack back when new element dominates.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) each element added/removed once from each deque	**O(n)**

Follow-up: shortest subarray with |max-min| > limit — same window, flipped condition.

Follow-up: Sliding Window Maximum (#239) — max deque only.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Heap | LeetCode #1488 | 27 of 40 | 27/40

Avoid Flood in The City

QUESTION

Your country has 10^9 lakes. Initially, all the lakes are empty, but when it rains over the nth lake, the nth lake becomes full of water. If it rains over a lake that is full of water, there will be a flood. Your goal is to avoid floods in any lake.

Given an integer array rains where:

CONSTRAINTS

- rains[i] > 0 means there will be rains over the rains[i] lake.
- rains[i] == 0 means there are no rains this day and you must choose one lake this day and dry it.

Return an array ans where:

- ans.length == rains.length
- ans[i] == -1 if rains[i] > 0.
- ans[i] is the lake you choose to dry in the ith day if rains[i] == 0.

If there are multiple valid answers return any of them. If it is impossible to avoid flood return an empty array.

Notice that if you chose to dry a full lake, it becomes empty, but if you chose to dry an empty lake, nothing changes.

Example 1

Input	`rains = [1,2,3,4]`
Output	`[-1,-1,-1,-1]`
Explanation	*After the first day full lakes are [1]* *After the second day full lakes are [1,2]* *After the third day full lakes are [1,2,3]* *After the fourth day full lakes are [1,2,3,4]* *There's no day to dry any lake and there is no flood in any lake.*

Example 2

Input	`rains = [1,2,0,0,2,1]`
Output	`[-1,-1,2,1,-1,-1]`
Explanation	*After the first day full lakes are [1]* *After the second day full lakes are [1,2]* *After the third day, we dry lake 2. Full lakes are [1]* *After the fourth day, we dry lake 1. There is no full lakes.* *After the fifth day, full lakes are [2].* *After the sixth day, full lakes are [1,2].* *It is easy that this scenario is flood-free.* *[-1,-1,1,2,-1,-1] is another acceptable scenario.*

QUESTION

Example 3

Input	`rains = [1,2,0,1,2]`
Output	`[]`
Explanation	*After the second day, full lakes are [1,2]. We have to dry one lake in the third day.* *After that, it will rain over lakes [1,2]. It's easy to prove that no matter which lake you choose to dry in the 3rd day, the other one will flood.*

- 1 <= rains.length <= 10^5
- 0 <= rains[i] <= 10^9

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

rains[i] > 0: it rains on lake rains[i]. rains[i] == 0: we can dry one lake.

If a lake overflows (it rains when already full), return []. Otherwise return drying schedule. Right?"

"A few quick questions:

For 0 days, we must choose which lake to dry. Return -1 for rain days (lake number), chosen lake for dry days?"

"Let me walk: rains=[1,2,3,4],no zeros -> no way to dry -> return [] ok? No: only fail if lake rains twice without drying. [1,2,3,4] no repeats -> [-1,-1,-1,-1] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for way in ways:
    # to assign a dry day to each lake that refills
    check that no lake floods
return a valid assignment
```

TIME	SPACE
exponential	O(n)

WHY IT IS SLOW

Tries every assignment. Greedy: keep a map from lake to the last day it rained and a sorted set of dry days; when a lake rains again, dry it on the earliest dry day after its previous rain: O(n log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Greedy with sorted-set for dry-day selection — O(n log n). Walk-through:"

Track last_full[lake] = day lake was last rained on.

`dry_days = sorted structure of available dry days.`

For each day:

If it rains on lake:

If lake is already full (in last_full): find the earliest dry day AFTER last_full[lake].

If no dry day available: return []. Use that dry day to drain the lake.

`last_full[lake] = today.`

If it's a dry day: add to dry_days pool.

Build ans[]: dry days get the lake they drain; rainy days get -1.

WHY earliest dry day after last rain: greedy — using the earliest possible dry day

preserves later ones for potentially urgent drains.

Time: O(n log n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
import bisect
class _1488_AvoidFloodInCity:
    def avoidFlood(self, rains):
        full = {}
        dry_days = []
        result = [-1] * len(rains)
        for i, lake in enumerate(rains):
            if lake == 0:
                dry_days.append(i)
            else:
                if lake in full:
                    idx = bisect.bisect_right(dry_days, fu
ll[lake])
                    if idx == len(dry_days):
                        return []
                    result[dry_days[idx]] = lake
                    dry_days.pop(idx)
                full[lake] = i
        for d in dry_days:
            result[d] = 1
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace how the heap property guides which element comes next."

rains=[1,2,0,0,2,1]: day0->full[1]=0. day1->full[2]=1. day2->dry_days=[2]. day3->dry_days=[2,3].

`day4(lake2): in full at 1. bisect_right([2,3],1)=0. result[2]=2. dry_days=[3].`

`day5(lake1): in full at 0. bisect_right([3],0)=0. result[3]=1. dry_days=[].`

-> [-1,-1,2,1,-1,-1] ok

Binary search finds first dry day AFTER the lake last rained

PHASE 5 TEST & DEBUG

WHY sorted set for dry days: binary search finds the next available dry day after a flood in O(log n) — greedy earliest-possible drying minimises flood conflicts.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n) bisect per double-rain event	**O(n)**

Follow-up: minimise number of dry days — different optimisation.

Follow-up: if drying capacity is k lakes per 0 day, extend dry_days to process k times.

MEDIUM | Heap | LeetCode #1514 | 28 of 40 | 28/40

Path with Maximum Probability

QUESTION

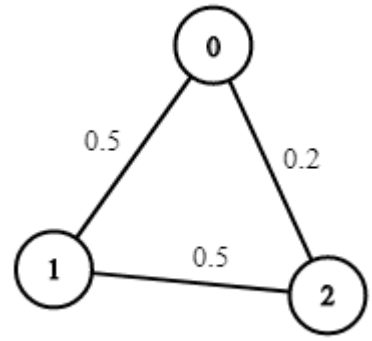
You are given an undirected weighted graph of n nodes (0-indexed), represented by an edge list where edges[i] = [a, b] is an undirected edge connecting the nodes a and b with a probability of success of traversing that edge succProb[i].

Given two nodes start and end, find the path with the maximum probability of success to go from start to end and return its success probability.

If there is no path from start to end, return 0. Your answer will be accepted if it differs from the correct answer by at most 1e-5.

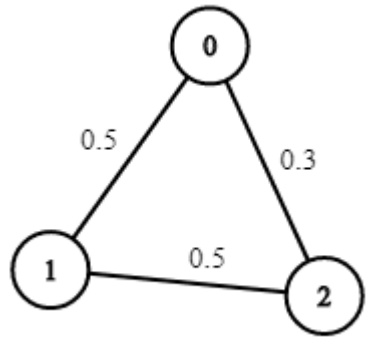
Example 1

Input	`n = 3, edges = [[0,1],[1,2],[0,2]], succProb = [0.5,0.5,0.2], start = 0, end = 2`
Output	`0.25000`
Explanation	*There are two paths from start to end, one having a probability of success = 0.2 and the other has 0.5 * 0.5 = 0.25.*



Example 2

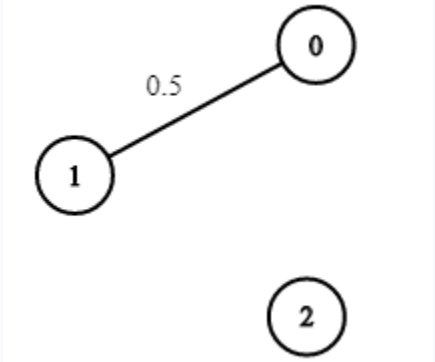
Input	`n = 3, edges = [[0,1],[1,2],[0,2]], succProb = [0.5,0.5,0.3], start = 0, end = 2`
Output	`0.30000`



QUESTION

Example 3

Input	`n = 3, edges = [[0,1]], succProb = [0.5], start = 0, end = 2`
Output	`0.00000`
Explanation	*There is no path between 0 and 2.*



CONSTRAINTS

- 2 <= n <= 10^4
- 0 <= start, end < n
- start != end
- 0 <= a, b < n
- a != b
- 0 <= succProb.length == edges.length <= 2*10^4
- 0 <= succProb[i] <= 1
- There is at most one edge between every two nodes.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given a graph with probabilities on edges, find the path from start to end with maximum probability. Right?"

"A few quick questions:

Probabilities are between 0 and 1; we multiply along a path. Return 0 if no path? Yes."

"Let me walk: n=3, edges=[[0,1],[1,2],[0,2]], succProb=[0.5,0.5,0.2], start=0, end=2 -> 0.25 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def dfs(node, probability):
    walk every simple path from start,
        multiplying edge probabilities
    best = max(best, reaches_end)
```

PHASE 2 BRUTE FORCE

TIME	SPACE
exponential	O(n)

WHY IT IS SLOW

The number of simple paths is exponential. Dijkstra with a max-heap on probability (multiplying instead of adding) or Bellman-Ford is O(E log V).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Dijkstra with MAX probability — already O((V+E) log V). Walk-through:"

Initialize prob[start] = 1.0; all others = 0.0.

Max-heap: [(-1.0, start)]. (Negate because Python has min-heap.)

Pop (-p, u). If -p < prob[u]: skip (stale).

For each neighbor v with edge probability w:

`new_prob = prob[u] * w`

If new_prob > prob[v]: prob[v] = new_prob; push (-new_prob, v).

WHY Dijkstra works for max probability: probabilities are between 0 and 1 (no increase

when multiplying). Greedy: the highest-probability state popped from the heap IS the best

possible (no path can improve it — multiplication can only decrease probability).

Return prob[end]. Time: O((V+E) log V). Space: O(V+E).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
import heapq
from collections import defaultdict
class _1514_PathWithMaxProbability:
    def maxProbability(self, n, edges, succProb, start, en
d):
        graph = defaultdict(list)
        for (u, v), p in zip(edges, succProb):
            graph[u].append((v, p))
            graph[v].append((u, p))
        prob = [0.0] * n
        prob[start] = 1.0
        heap = [(-1.0, start)]
        while heap:
            neg_p, node = heapq.heappop(heap)
            cur_prob = -neg_p
            if cur_prob < prob[node]:
                continue
            for neighbor, edge_prob in graph[node]:
                new_prob = cur_prob * edge_prob
                if new_prob > prob[neighbor]:
                    prob[neighbor] = new_prob
                    heapq.heappush(heap, (-new_prob, neigh
bor))
        return prob[end]
```

PHASE 5 TEST & DEBUG

"Let me trace maxProbability(n=3, edges=[[0,1],[1,2],[0,2]], succProb=[0.5,0.5,0.2], src=0, dst=2)."

Bellman-Ford / Dijkstra on undirected graph with edge probabilities

Max probability: treat as shortest path but multiply probs (maximize product)

Dijkstra with max-heap: prob[0]=1.0; prob[1]=prob[2]=0

Pop (1.0,0): neighbors: 1->prob*0.5=0.5, 2->prob*0.2=0.2

Pop (0.5,1): neighbors: 0->already, 2->0.5*0.5=0.25 > 0.2 -> update prob[2]=0.25

Pop (0.25,2): reached dst=2 -> return 0.25 ok

WHY Dijkstra max-heap: maximize product by exploring highest-probability paths first.

Dijkstra BFS (max-prob): heap=[(1.0,src)]. visited={}. Pop (prob,v): if v in visited skip. Push (prob*edge_prob,neighbor). visited[v]=True on pop. Until dst popped.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O((V+E) log V)	O(V+E)

Bellman-Ford alternative: O(V*E) — usable if probabilities can be > 1 (not possible here).

Follow-up: Network Delay Time (#743) — same Dijkstra, sum weights instead of multiply.

Follow-up: if we want the actual path, store parent pointers during relaxation.

MEDIUM | Heap | LeetCode #1631 | 29 of 40 | 29/40

Path with Minimum Effort

QUESTION

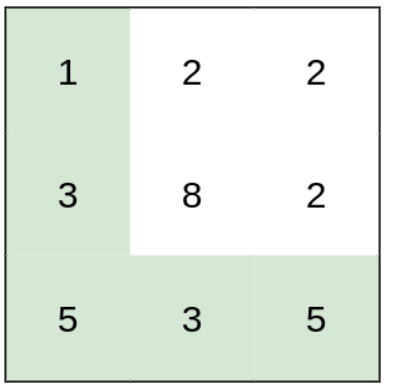
You are a hiker preparing for an upcoming hike. You are given heights, a 2D array of size rows x columns, where heights[row][col] represents the height of cell (row, col). You are situated in the top-left cell, (0, 0), and you hope to travel to the bottom-right cell, (rows-1, columns-1) (i.e., 0-indexed). You can move up, down, left, or right, and you wish to find a route that requires the minimum effort.

A route's effort is the maximum absolute difference in heights between two consecutive cells of the route.

Return the minimum effort required to travel from the top-left cell to the bottom-right cell.

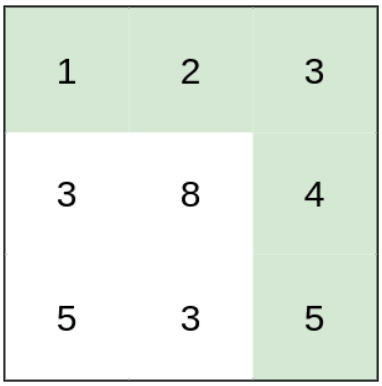
Example 1

Input	`heights = [[1,2,2],[3,8,2],[5,3,5]]`
Output	`2`
Explanation	*The route of [1,3,5,3,5] has a maximum absolute difference of 2 in consecutive cells. This is better than the route of [1,2,2,2,5], where the maximum absolute difference is 3.*



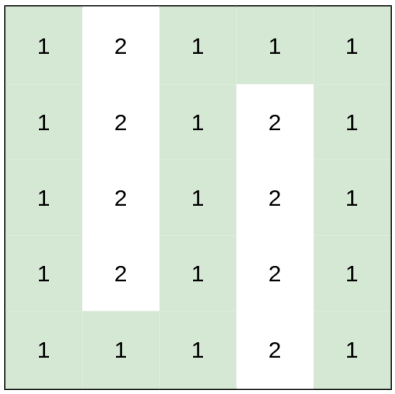
Example 2

Input	`heights = [[1,2,3],[3,8,4],[5,3,5]]`
Output	`1`
Explanation	*The route of [1,2,3,4,5] has a maximum absolute difference of 1 in consecutive cells, which is better than route [1,3,5,3,5].*



Example 3

Input	`heights = [[1,2,1,1,1],[1,2,1,2,1],[1,2,1,2,1],[1,2,1,2,1],[1,1,1,2,1]]`
Output	`0`
Explanation	*This route does not require any effort.*



CONSTRAINTS

- rows == heights.length
- columns == heights[i].length
- 1 <= rows, columns <= 100
- 1 <= heights[i][j] <= 10^6

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the path from (0,0) to (m-1,n-1) that minimises the maximum absolute difference between adjacent cells. Right?"

"A few quick questions:

4-directional movement? Yes. Minimise the bottleneck (maximum single-step effort)? Yes."

"Let me walk: heights=[[1,2,2],[3,8,2],[5,3,5]] -> path 1->2->2->3->5 -> max_diff=max(1,0,1,2)=2 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def dfs(cell, effort):
    move to each neighbour, tracking the largest
        height difference so far, along every
        simple path
    best = min(best, effort)
```

TIME	SPACE
O(4^(m * n))	O(m * n)

WHY IT IS SLOW

Explores every path. Dijkstra with a min-heap on the largest difference so far, or binary search on the effort with one BFS per check, is O(m * n log(m * n)).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Dijkstra for minimum effort — already O(mn log(mn)). Walk-through:"

effort[r][c] = minimum possible max-difference along any path from (0,0) to (r,c).

Initialize effort[0][0]=0; all others=infinity. Min-heap: [(0, 0, 0)].

Pop (e, r, c). If e > effort[r][c]: skip.

For each of 4 neighbors (nr, nc):

step_effort = abs(heights[nr][nc] - heights[r][c])

new_effort = max(e, step_effort) <- bottleneck = max step seen on this path

If new_effort < effort[nr][nc]:

effort[nr][nc] = new_effort; push.

WHY Dijkstra with max bottleneck: we want to minimize the WORST single step.

Max-in-path acts like edge weight in Dijkstra — greedy expansion finds optimal.

Return effort[m-1][n-1]. Time: O(mn log(mn)). Space: O(mn).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
import heapq
class _1631_PathWithMinimumEffort:
    def minimumEffortPath(self, heights):
        m, n = len(heights), len(heights[0])
        effort = [[float('inf')] * n for _ in range(m)]
        effort[0][0] = 0
        heap = [(0, 0, 0)]
        while heap:
            e, r, c = heapq.heappop(heap)
            if r == m-1 and c == n-1:
                return e
            if e > effort[r][c]:
                continue
            for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
                nr, nc = r+dr, c+dc
                if 0 <= nr < m and 0 <= nc < n:
                    new_effort = max(e, abs(heights[nr][nc] - heights[r][c]))
                    if new_effort < effort[nr][nc]:
                        effort[nr][nc] = new_effort
                        heapq.heappush(heap, (new_effort, nr, nc))
        return effort[m-1][n-1]
```

PHASE 5 TEST & DEBUG

"Let me trace minimumEffortPath([[1,2,2],[3,8,2],[5,3,5]])."

Dijkstra: cost(path) = max(|heights[i]-heights[j]|) over consecutive cells

Start (0,0)=1. Heap=[(0,0,0)] (effort,r,c). dist[0][0]=0.

Pop(0,0,0): neighbors (0,1)=2:effort=|1-2|=1; (1,0)=3:effort=2

Pop(1,0,1)=2: neighbors (0,2)=2:max(1,0)=1; (1,1)=8:max(1,|2-8|)=6

Pop(1,0,2)=2: neighbor (1,2)=2:max(1,0)=1

Pop(1,1,2)=2: neighbor (2,2)=5:max(1,|2-5|)=3

Pop(2,1,0)=3: neighbor (2,0)=5:max(2,2)=2; (1,1)=8:max(2,5)=5

Eventually reach (2,2): effort=2 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(mn log(mn))	O(mn)

Binary search + BFS/DFS is O(mn log(max_height)) — similar complexity.

Follow-up: Path With Maximum Minimum Value (#1102) — max instead of min bottleneck.

Follow-up: Swim in Rising Water (#778) — same Dijkstra pattern.

MEDIUM | Heap | LeetCode #1642 | 30 of 40 | 30/40

Furthest Building You Can Reach

QUESTION

You are given an integer array heights representing the heights of buildings, some bricks, and some ladders.

You start your journey from building 0 and move to the next building by possibly using bricks or ladders.

While moving from building i to building i+1 (0-indexed),

CONSTRAINTS

- If the current building's height is greater than or equal to the next building's height, you do not need a ladder or bricks.
- If the current building's height is less than the next building's height, you can either use one ladder or (h[i+1] - h[i]) bricks.

Return the furthest building index (0-indexed) you can reach if you use the given ladders and bricks optimally.

Example 1

Input	`heights = [4,2,7,6,9,14,12], bricks = 5, ladders = 1`
Output	`4`
Explanation	*Starting at building 0, you can follow these steps:* *- Go to building 1 without using ladders nor bricks since 4 >= 2.* *- Go to building 2 using 5 bricks. You must use either bricks or ladders because 2 < 7.* *- Go to building 3 without using ladders nor bricks since 7 >= 6.* *- Go to building 4 using your only ladder. You must use either bricks or ladders because 6 < 9.* *It is impossible to go beyond building 4 because you do not have any more bricks or ladders.*



Example 2

Input	`heights = [4,12,2,7,3,18,20,3,19], bricks = 10, ladders = 2`
Output	`7`

Example 3

Input	`heights = [14,3,19,3], bricks = 17, ladders = 0`
Output	`3`

- $1 <= heights.length <= 10^5$
- $1 <= heights[i] <= 10^6$
- $0 <= bricks <= 10^9$
- 0 <= ladders <= heights.length

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given building heights and a limited number of bricks and ladders, reach the furthest building. Ladders for tall jumps, bricks for small ones. Right?"

"A few quick questions:

Only move forward (no backtracking)? Yes. Jumps are positive differences only? Yes."

"Let me walk: heights=[4,2,7,6,9,14,12], bricks=5, ladders=1 -> 4 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for choice in choices:
    # of which climbs get a ladder and which get
        bricks
    simulate
return the furthest building reached
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

Tries every assignment. Use a ladder for every climb at first, keeping the ladders' climbs in a min-heap; when out of ladders, pay bricks for the smallest climb in the heap: O(n log L).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is optimal allocation.

Greedy: always use ladders for the tallest jumps seen so far.

Use a min-heap of size ladders: if current diff > min in heap, use ladder there instead.

The min in heap is the smallest 'ladder climb' — can be replaced with bricks.

PHASE 3 OPTIMIZE

Time: O(n log ladders). Space: O(ladders).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
import heapq
class _1642_FurthestBuildingYouCanReach:
    def furthestBuilding(self, heights, bricks, ladders):
        heap = []
        for i in range(len(heights) - 1):
            diff = heights[i+1] - heights[i]
            if diff <= 0:
                continue
            heapq.heappush(heap, diff)
            if len(heap) > ladders:
                bricks -= heapq.heappop(heap)
            if bricks < 0:
                return i
        return len(heights) - 1
```

PHASE 5 TEST & DEBUG

"Let me trace furthestBuilding([4,2,7,6,9,14,12], bricks=5, ladders=1)."

Greedy with min-heap: use ladders for largest climbs, bricks for smaller

4->2: going down (diff=-2), no resource needed

2->7: diff=5. heap=[5], ladders used=1

7->6: going down, skip

6->9: diff=3. heap=[3,5] but ladders=1 already used. Pop min=3, use bricks 3. bricks=5-3=2. heap=[5]

9->14: diff=5. heap=[5,5]. Pop min=5. bricks=2-5<0! -> use brick: 2-5<0 -> return index 4 ok?

Actually: 9->14 diff=5. heap=[5]. ladders=0. bricks=2-5<0 -> return 4 (can't proceed) ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log ladders)	**O(ladders)** heap

Follow-up: if ladders and bricks can be used in any order (no restriction), same greedy.

Follow-up: Minimum Cost to Connect Sticks (#1167) — different heap application.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Heap | LeetCode #1696 | 31 of 40 | 31/40

Jump Game VI

QUESTION

You are given a 0-indexed integer array nums and an integer k.

You are initially standing at index 0. In one move, you can jump at most k steps forward without going outside the boundaries of the array. That is, you can jump from index i to any index in the range [i + 1, min(n - 1, i + k)] inclusive.

You want to reach the last index of the array (index n - 1). Your score is the sum of all nums[j] for each index j you visited in the array.

Return the maximum score you can get.

Example 1

Input	`nums = [ 1 , -1 ,-2, 4 ,-7, 3 ], k = 2`
Output	`7`
Explanation	*You can choose your jumps forming the subsequence [1,-1,4,3] (underlined above). The sum is 7.*

Example 2

Input	`nums = [ 10 ,-5,-2, 4 ,0, 3 ], k = 3`
Output	`17`
Explanation	*You can choose your jumps forming the subsequence [10,4,3] (underlined above). The sum is 17.*

Example 3

Input	`nums = [1,-5,-20,4,-1,3,-6,-3], k = 2`
Output	`0`

CONSTRAINTS

- 1 <= nums.length, k <= 10^5
- 10^4 <= nums[i] <= 10^4

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Jump Game: from index i, jump to any index in [i+1, i+k]. Maximise sum of visited indices.

`dp[i] = max score to reach index i. Right?"`

"A few quick questions:

Start at index 0 with score nums[0]? Yes. Must reach index n-1? No — maximise."

"Let me walk: nums=[1,-1,-2,4,-7,3], k=2 -> max path: 1,-1,4,3 = 7 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def go(i):
    if i is the last index:
        return nums[i]
    return nums[i] + the largest go(i + step)
        over every step from 1 to k
```

TIME	SPACE
O(k^n)	O(n)

WHY IT IS SLOW

k branches per index with repeated indices. dp[i] = nums[i] + max of dp in the next k positions; a monotonic deque keeps that window maximum in O(1), so O(n) overall.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Monotone deque sliding window DP — O(n). Walk-through:"

`dp[i] = max score to reach index i.`

`dp[0] = nums[0]. Deque stores indices with decreasing dp values.`

For i in 1..n-1:

Remove indices outside window: while deque and deque[0] < i-k: popleft.

`dp[i] = dp[deque[0]] + nums[i] <- best reachable predecessor + current.`

Remove worse candidates: while deque and dp[deque[-1]] <= dp[i]: pop.

append i to deque.

WHY deque front = best predecessor: deque maintained in decreasing dp order, oldest first.

Front is the index with maximum dp within the window.

WHY pop back when dp[i] >= dp[back]: back can never be the best predecessor for future

elements (dp[i] is both larger AND more recent -> always preferable).

Time: O(n). Space: O(k).

IN YOUR APP | Expose | Editorial | Community

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
from collections import deque
class _1696_JumpGameVI:
    def maxResult(self, nums, k):
        n = len(nums)
        dp = [0] * n
        dp[0] = nums[0]
        dq = deque([0])
        for i in range(1, n):
            while dq and dq[0] < i - k:
                dq.popleft()
            dp[i] = nums[i] + dp[dq[0]]
            while dq and dp[dq[-1]] <= dp[i]:
                dq.pop()
            dq.append(i)
        return dp[-1]
```

PHASE 5 TEST & DEBUG

"Let me trace the DP array and show how the deque front always holds the best in-window predecessor."

nums=[1,-1,-2,4,-7,3], k=2: dp[0]=1, dq=[0].

i=1: dq[0]=0>=1-2=-1 ok. dp[1]=-1+1=0. dq=[0,1].

i=2: dq[0]=0>=0 ok. dp[2]=-2+1=-1. dp[1]=0>-1->keep dq=[0,1,2]? No: dp[2]=-1<dp[1]=0->keep. dq=[0,1,2].

i=3: dq[0]=0<3-2=1->pop. dq=[1,2]. dp[3]=4+max(dp[1],dp[2])=4+0=4. dp[2]=-1<4->pop. dp[1]=0<4->pop. dq=[3].

i=4: dq[0]=3>=2 ok. dp[4]=-7+4=-3. dp[3]=4>-3->keep. dq=[3,4].

i=5: dq[0]=3>=3 ok. dp[5]=3+4=7. dq: dp[4]=-3<7->pop. dp[3]=4<7->pop. dq=[5]. -> 7 ok

Deque maintains decreasing dp values; front is always the maximum in window

WHY monotonic deque (not a heap): dp[i] = nums[i] + max(dp[i-k..i-1]); deque front is always the best in-window predecessor — same O(1) amortized trick as LC 1425.

Monotonic deque as stack: stack=[best_dp_idx,...]. Pop back if dp[i]<=dp[stack.back()]. Pop front if stack.front() out of window. stack[front]=index of max dp in window.

Monotonic deque=[0,1,2] (indices of decreasing dp values). deque front=max dp in window. popleft() when front index out of window [i-k,i]. Pop back when dp[i]>=dp[back]. deque=[0]->[0,1]->...

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(k)** for the deque

Follow-up: Jump Game VII (#1871) — binary constraints (0/1 grid), BFS approach.

Follow-up: Constrained Subsequence Sum (#1425) — same sliding window max DP.

MEDIUM | Heap | LeetCode #1834 | 32 of 40 | 32/40

Single-Threaded CPU

QUESTION

You are given n tasks labeled from 0 to n - 1 represented by a 2D integer array tasks, where tasks[i] = [enqueueTime$_i$, processingTime$_i$] means that the ith task will be available to process at enqueueTime$_i$ and will take processingTime$_i$ to finish processing.

You have a single-threaded CPU that can process at most one task at a time and will act in the following way:

CONSTRAINTS

- If the CPU is idle and there are no available tasks to process, the CPU remains idle.
- If the CPU is idle and there are available tasks, the CPU will choose the one with the shortest processing time. If multiple tasks have the same shortest processing time, it will choose the task with the smallest index.
- Once a task is started, the CPU will process the entire task without stopping.
- The CPU can finish a task then start a new one instantly.

Return the order in which the CPU will process the tasks.

Example 1

Input	`tasks = [[1,2],[2,4],[3,2],[4,1]]`
Output	`[0,2,3,1]`
Explanation	*The events go as follows:* *- At time = 1, task 0 is available to process. Available tasks = {0}.* *- Also at time = 1, the idle CPU starts processing task 0. Available tasks = {}.* *- At time = 2, task 1 is available to process. Available tasks = {1}.* *- At time = 3, task 2 is available to process. Available tasks = {1, 2}.* *- Also at time = 3, the CPU finishes task 0 and starts processing task 2 as it is the shortest. Available tasks = {1}.* *- At time = 4, task 3 is available to process. Available tasks = {1, 3}.* *- At time = 5, the CPU finishes task 2 and starts processing task 3 as it is the shortest. Available tasks = {1}.* *- At time = 6, the CPU finishes task 3 and starts processing task 1. Available tasks = {}.* *- At time = 10, the CPU finishes task 1 and becomes idle.*

QUESTION

Example 2

Input	`tasks = [[7,10],[7,12],[7,5],[7,4],[7,2]]`

Output	`[4,3,2,0,1]`
	`Explanation:`
	`The events go as follows:`
	`- At time = 7, all the tasks become available. Available tasks = {0,1,2,3,4}.`
	`- Also at time = 7, the idle CPU starts processing task 4. Available tasks = {0,1,2,3}.`
	`- At time = 9, the CPU finishes task 4 and starts processing task 3. Available tasks = {0,1,2}.`
	`- At time = 13, the CPU finishes task 3 and starts processing task 2. Available tasks = {0,1}.`
	`- At time = 18, the CPU finishes task 2 and starts processing task 0. Available tasks = {1}.`
	`- At time = 28, the CPU finishes task 0 and starts processing task 1. Available tasks = {}.`
	`- At time = 40, the CPU finishes task 1 and becomes idle.`

- tasks.length == n
- 1 <= n <= 10^5
- 1 <= enqueueTime$_i$, processingTime$_i$ <= 10^9

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Single-threaded CPU. Tasks have enqueue time and processing time.

At each step, pick the shortest task available (tie: smallest index). Return task order. Right?"

"A few quick questions:

If CPU is idle and no tasks queued, advance time to next task's enqueue time? Yes."

"Let me walk: tasks=[[1,2],[2,4],[3,2],[4,1]] -> [0,2,3,1] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
time = 0
while True:
    among the tasks that have arrived and are not
        done, scan for the shortest one (smallest
        index on ties), run it, advance the time
        (jump to the next arrival if idle)
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

A scan of all tasks at every step. Sort tasks by enqueue time and keep a min-heap of available tasks keyed by (processing time, index): O(n log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Two heaps: available CPUs + enqueued tasks — O(n log n). Walk-through:"

`available = max-heap of free CPU IDs (negate for min-heap behavior: use min CPU ID).`

Wait, problem: single-threaded CPU — one task at a time.

Sort tasks by enqueue time. Two heaps: available_tasks (min processing time), time tracking.

`curr_time = 0.`

While tasks remain or task heap non-empty:

If task heap empty: curr_time = next task's enqueue time (jump forward).

Add all tasks with enqueue_time <= curr_time to task heap (min-heap by processing time).

Pick task with shortest processing time: (proc_time, idx) = heappop(task_heap).

`result.append(idx). curr_time += proc_time.`

WHY jump time forward: no task available -> CPU is idle -> skip to next available task.

WHY min-heap on processing time: ties broken by shorter tasks (implicit: then by index).

Time: O(n log n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
import heapq
class _1834_SingleThreadedCPU:
    def getOrder(self, tasks):
        sorted_tasks = sorted(enumerate(tasks), key=lambda
 x: x[1][0])
        heap = []
        current_time = 0
        result = []
        i = 0
        while heap or i < len(sorted_tasks):
            if not heap:
                current_time = max(current_time, sorted_ta
sks[i][1][0])
            while i < len(sorted_tasks) and sorted_tasks[i
][1][0] <= current_time:
                idx, (enqueue, process) = sorted_tasks[i]
                heapq.heappush(heap, (process, idx))
                i += 1
            process_time, idx = heapq.heappop(heap)
            result.append(idx)
            current_time += process_time
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace how the heap property guides which element comes next."

tasks=[[1,2],[2,4],[3,2],[4,1]]: sorted by enqueue: [(0,1,2),(1,2,4),(2,3,2),(3,4,1)].

t=1: add (2,0). pop(2,0)->result=[0]. t=3. t=3: add (4,1),(2,2). pop(2,2)->result=[0,2]. t=5.

t=5: add (1,3). pop(1,3)->result=[0,2,3]. t=6. pop(4,1)->result=[0,2,3,1] ok

Heap (process_time, idx) ensures shortest-then-smallest-index ordering

WHY two heaps (free + busy): at each task time, free all finished processors first, then assign to lowest-index free one or earliest-finishing busy one — O(n log k).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n)	O(n)

Follow-up: Parallel CPU scheduling — extend to k CPUs, use k-element heap.

Follow-up: Process Tasks Using Servers (#1882) — assign tasks to servers by availability.

MEDIUM | Heap | LeetCode #1882 | 33 of 40 | 33/40

Process Tasks Using Servers

QUESTION

You are given two 0-indexed integer arrays servers and tasks of lengths n and m respectively. servers[i] is the weight of the ith server, and tasks[j] is the time needed to process the jth task in seconds.

Tasks are assigned to the servers using a task queue. Initially, all servers are free, and the queue is empty.

At second j, the jth task is inserted into the queue (starting with the 0th task being inserted at second 0). As long as there are free servers and the queue is not empty, the task in the front of the queue will be assigned to a free server with the smallest weight, and in case of a tie, it is assigned to a free server with the smallest index.

If there are no free servers and the queue is not empty, we wait until a server becomes free and immediately assign the next task. If multiple servers become free at the same time, then multiple tasks from the queue will be assigned in order of insertion following the weight and index priorities above.

A server that is assigned task j at second t will be free again at second t + tasks[j].

Build an array ans of length m, where ans[j] is the index of the server the jth task will be assigned to.

Return the array ans.

Example 1

Input	`servers = [3,3,2], tasks = [1,2,3,2,1,2]`
Output	`[2,2,0,2,1,2]`
Explanation	*Events in chronological order go as follows:* *- At second 0, task 0 is added and processed using server 2 until second 1.* *- At second 1, server 2 becomes free. Task 1 is added and processed using server 2 until second 3.* *- At second 2, task 2 is added and processed using server 0 until second 5.* *- At second 3, server 2 becomes free. Task 3 is added and processed using server 2 until second 5.* *- At second 4, task 4 is added and processed using server 1 until second 5.* *- At second 5, all servers become free. Task 5 is added and processed using server 2 until second 7.*

QUESTION

Example 2

Input	`servers = [5,1,4,3,2], tasks = [2,1,2,4,5,2,1]`
Output	`[1,4,1,4,1,3,2]`
Explanation	*Events in chronological order go as follows:* *- At second 0, task 0 is added and processed using server 1 until second 2.* *- At second 1, task 1 is added and processed using server 4 until second 2.* *- At second 2, servers 1 and 4 become free. Task 2 is added and processed using server 1 until second 4.* *- At second 3, task 3 is added and processed using server 4 until second 7.* *- At second 4, server 1 becomes free. Task 4 is added and processed using server 1 until second 9.* *- At second 5, task 5 is added and processed using server 3 until second 7.* *- At second 6, task 6 is added and processed using server 2 until second 7.*

CONSTRAINTS

- servers.length == n
- tasks.length == m
- 1 <= n, m <= $2 * 10^5$
- 1 <= servers[i], tasks[j] <= $2 * 10^5$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Assign each task (arriving at second i) to the free server with smallest weight (tie: smallest index).

If none free, wait until one finishes. Return which server handles each task. Right?"

"A few quick questions:

Tasks arrive one per second (0,1,2,...,n-1)? Yes. Server processes for tasks[i] seconds."

"Let me walk: servers=[3,3,2], tasks=[1,2,3,2,1,2] -> [2,2,0,2,1,2] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for task in order:
    scan all servers for the free one with the
        smallest weight (index on ties), or wait
        for the earliest one to free up
```

TIME	SPACE
O(m * n)	O(n)

PHASE 2 BRUTE FORCE

WHY IT IS SLOW

A scan of all servers per task. Two heaps, free servers by (weight, index) and busy servers by (free time, weight, index), give O((n + m) log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Two heaps: free servers + busy servers — O(n log n). Walk-through:"

free = min-heap of server IDs (initially all servers: [0,1,...,k-1]).

busy = min-heap of (finish_time, server_id).

For each task i with arrival_time = i//1 (tasks arrive sequentially at time i):

Wait, tasks arrive at times 0,1,2,... Actually tasks[i] is the processing time.

current_time = i (task i arrives at time i).

Free up servers whose finish_time <= current_time:

While busy and busy[0][0] <= current_time: heappush(free, heappop(busy)[1]).

If free empty: wait for next server — heappop busy, free that server, advance time.

Assign task to server with smallest ID: server = heappop(free).

result[server] += 1. Push (current_time + tasks[i], server) to busy.

Return result.index(max(result)). Time: O(n log k). Space: O(k)."

Key insight: two heaps (free servers keyed by weight/index, busy servers keyed by finish time) decouple availability tracking — at each task, we first free finished servers then assign to the best available, all O(log k) per step.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
import heapq
class _1882_ProcessTasksUsingServers:
    def assignTasks(self, servers, tasks):
        free = [(w, i) for i, w in enumerate(servers)]
        heapq.heapify(free)
        busy = []
        result = []
        for t, task_time in enumerate(tasks):
            current_time = max(t, busy[0][0] if busy and n
ot free else t)
            while busy and busy[0][0] <= current_time:
                finish, w, i = heapq.heappop(busy)
                heapq.heappush(free, (w, i))
            if not free:
                finish, w, i = heapq.heappop(busy)
                current_time = finish
                heapq.heappush(free, (w, i))
                while busy and busy[0][0] <= current_time:
                    f, w2, i2 = heapq.heappop(busy)
                    heapq.heappush(free, (w2, i2))
            w, i = heapq.heappop(free)
            result.append(i)
            heapq.heappush(busy, (current_time + task_time
, w, i))
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace assignTasks(servers=[3,3,2],tasks=[1,2,3,2,1,2])."

Simulate: free servers heap (by weight, index). Busy servers heap (by finish_time, weight, index).

Free: [(2,2),(3,0),(3,1)].

t=0,task=1: pop (2,2). Server 2 handles task. Busy:[(0+1=1,2,2)]. result[0]=2.

t=1,task=2: release busy servers finishing <=1: (1,2,2)->free=(2,2). Pop (2,2). result[1]=2.

t=2,task=3: release none. Pop (2,2)? already used. Free=[(3,0),(3,1)]. Pop (3,0). result[2]=0.

t=3,task=2: free server: pop (3,1). result[3]=1. t=4,task=1: release (2,0)->free. pop (2,0)... continue.

Return [2,2,0,2,1,2] ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log m)	**O(m)**
n tasks, heap ops O(log m) each	heaps

Follow-up: Single-Threaded CPU (#1834) — tasks assigned in order; single queue.

Follow-up: if servers have capacity > 1, extend busy heap to track slot counts.

IN YOUR APP | Expose | Community

MEDIUM | Heap | LeetCode #1985 | 34 of 40 | 34/40

Find the Kth Largest Integer in the Array

QUESTION

You are given an array of strings nums and an integer k. Each string in nums represents an integer without leading zeros.

Return the string that represents the kth largest integer in nums.

Note: Duplicate numbers should be counted distinctly. For example, if nums is ["1","2","2"], "2" is the first largest integer, "2" is the second-largest integer, and "1" is the third-largest integer.

Example 1

Input	`nums = ["3","6","7","10"], k = 4`
Output	`"3"`
Explanation	*The numbers in nums sorted in non-decreasing order are ["3","6","7","10"]. The 4th largest integer in nums is "3".*

Example 2

Input	`nums = ["2","21","12","1"], k = 3`
Output	`"2"`
Explanation	*The numbers in nums sorted in non-decreasing order are ["1","2","12","21"]. The 3rd largest integer in nums is "2".*

Example 3

Input	`nums = ["0","0"], k = 2`
Output	`"0"`
Explanation	*The numbers in nums sorted in non-decreasing order are ["0","0"]. The 2nd largest integer in nums is "0".*

CONSTRAINTS

- $1 <= k <= nums.length <= 10^4$
- 1 <= nums[i].length <= 100
- nums[i] consists of only digits.
- nums[i] will not have any leading zeros.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given an array of numeric strings, find the kth largest integer (by numeric value).

Return as a string. Right?"

"A few quick questions:

Strings represent non-negative integers with no leading zeros? Yes."

PHASE 1 CLARIFY

"Let me walk: nums=['3','6','7','10'], k=4 -> 4th largest = '3' ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
convert every string to a big integer
sort the integers in descending order
return the k-th
```

TIME	SPACE
O(n * L * log n)	O(n)

WHY IT IS SLOW

Sorts everything and compares long numbers. Compare strings by (length, then lexicographic order); a heap of size k or quickselect with that comparison is O(n * L * log k) or O(n * L) on average.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Sort as integers, return k-th largest — O(n log n). Walk-through:"

Convert strings to integers for correct numeric comparison (not lexicographic).

Sort descending. Return nums[k-1].

WHY convert to int: '100' > '99' numerically but '99' > '100' lexicographically

(comparing char '9' vs '1'). Must compare as integers.

Alternative O(n) QuickSelect: partial sort, stop when pivot is at position n-k.

Picks answer without fully sorting -> O(n) average.

Time: O(n log n) sort or O(n) avg QuickSelect. Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
import heapq
class _1985_FindKthLargestInteger:
    def kthLargestNumber(self, nums, k):
        def cmp_key(s):
            return (len(s), s)
        heap = []
        for num in nums:
            heapq.heappush(heap, cmp_key(num))
            if len(heap) > k:
                heapq.heappop(heap)
        return heap[0][1]
```

PHASE 5 TEST & DEBUG

"Let me trace kthLargestNumber(['3','6','7','10'], k=4)."

These are string representations of integers; sort by numeric value

Sorted numerically: '3','6','7','10' -> 4th largest=3 ok (largest=10,6,7,3)

QuickSelect: partition around pivot comparing by numeric value

WHY custom comparator: '10'>'9' lexicographically but 10>9 numerically.

Comparator: compare int(a) vs int(b) or compare by length then lexicographically for same length.

kth largest = kth from end in sorted = quickselect in O(n) expected.

WHY compare as integers (len then lex): '100' > '99' by length, '9' > '10' lex but not numerically.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log k)	**O(k)** heap

Sort approach: O(n log n) — simpler to write for an interview.

Follow-up: Kth Largest Element in Array (#215) — integers not strings; use QuickSelect.

Follow-up: if strings can have leading zeros, strip them before comparison.

IN YOUR APP | Expose | Community

MEDIUM | Heap | LeetCode #2034 | 35 of 40 | 35/40

Stock Price Fluctuation

QUESTION

You are given a stream of records about a particular stock. Each record contains a timestamp and the corresponding price of the stock at that timestamp.

Unfortunately due to the volatile nature of the stock market, the records do not come in order. Even worse, some records may be incorrect. Another record with the same timestamp may appear later in the stream correcting the price of the previous wrong record.

Design an algorithm that:

CONSTRAINTS

- Updates the price of the stock at a particular timestamp, correcting the price from any previous records at the timestamp.
- Finds the latest price of the stock based on the current records. The latest price is the price at the latest timestamp recorded.
- Finds the maximum price the stock has been based on the current records.
- Finds the minimum price the stock has been based on the current records.

Implement the StockPrice class:

- StockPrice() Initializes the object with no price records.
- void update(int timestamp, int price) Updates the price of the stock at the given timestamp.
- int current() Returns the latest price of the stock.
- int maximum() Returns the maximum price of the stock.
- int minimum() Returns the minimum price of the stock.
- 1 <= timestamp, price <= 10^9
- At most 10^5 calls will be made in total to update, current, maximum, and minimum.
- current, maximum, and minimum will be called only after update has been called at least once.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Stock price updates with timestamps; support: update(timestamp, price), current(), maximum(), minimum(). Right?"

"A few quick questions:

Timestamps can update existing prices (corrections)? Yes. current() returns price at latest timestamp."

"Let me walk: update(1,10),update(2,5),current()->5,maximum()->10,minimum()->5 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
keep every (timestamp, price) record
current:
    the price at the largest timestamp
    maximum / minimum: scan every timestamp's
        latest price
```

TIME	SPACE
O(n) per query	O(n)

WHY IT IS SLOW

Scans all records per query. A hash map timestamp -> price plus a sorted structure or two heaps with lazy deletion (ignore entries whose price no longer matches the map) gives O(log n) per update and query.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"HashMap timestamp->price + SortedList of prices — O(log n) per op. Walk-through:"

time_to_price = {}. prices = SortedList().

update(timestamp, price):

If timestamp in time_to_price:
prices.remove(time_to_price[timestamp]). <- remove old price

time_to_price[timestamp] = price. prices.add(price).

current(): return time_to_price[max(time_to_price)].

maximum(): return prices[-1]. <- largest in sorted list

minimum(): return prices[0]. <- smallest in sorted list

WHY SortedList for prices: O(log n) insert/delete and O(1) min/max access.

WHY remove old price on update: same timestamp = correction -> must reflect updated price.

Time: O(log n) per operation. Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
from sortedcontainers import SortedList
class _2034_StockPriceFluctuation:
    def __init__(self):
        self._ts_to_price = {}
        self._prices = SortedList()
        self._max_ts = 0

    def update(self, timestamp, price):
        if timestamp in self._ts_to_price:
            self._prices.remove(self._ts_to_price[timestamp])
        self._ts_to_price[timestamp] = price
```

```
        self._prices.add(price)
        self._max_ts = max(self._max_ts, timestamp)

    def current(self): return self._ts_to_price[self._max_
ts]
    def maximum(self): return self._prices[-1]
    def minimum(self): return self._prices[0]
```

PHASE 5 TEST & DEBUG

"Let me trace StockPrice: update(1,10), update(2,5), update(1,3), current(), maximum(), minimum()."

Store: time->price map. Sorted set/BST for all current prices.

update(1,10): prices={1:10}. sorted_prices=[10]

update(2,5): prices={1:10,2:5}. sorted_prices=[5,10]

update(1,3): prices={1:3,2:5}. Remove old price 10, add 3. sorted_prices=[3,5]

current(): latest time=2, price=5 ok

maximum(): max of sorted_prices=5 ok

minimum(): min=3 ok

WHY sorted multiset: O(log n) update/query for max/min; time map for corrections.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(log n) per update, O(1) for current/max/min	**O(n)**

Without sortedcontainers: use two heaps (max and min) with lazy deletion.

Follow-up: return kth highest price — SortedList[-k] is O(1).

Follow-up: moving average over last k updates — deque of last k prices.

MEDIUM | Heap | LeetCode #2054 | 36 of 40 | 36/40

Two Best Non-Overlapping Events

QUESTION

You are given a 0-indexed 2D integer array of events where events[i] = [startTime$_i$, endTime$_i$, value$_i$]. The ith event starts at startTime$_i$ and ends at endTime$_i$, and if you attend this event, you will receive a value of value$_i$. You can choose at most two non-overlapping events to attend such that the sum of their values is maximized.

Return this maximum sum.

Note that the start time and end time is inclusive: that is, you cannot attend two events where one of them starts and the other ends at the same time. More specifically, if you attend an event with end time t, the next event must start at or after t + 1.

Example 1

Input	`events = [[1,3,2],[4,5,2],[2,4,3]]`
Output	**4**
Explanation	*Choose the green events, 0 and 1 for a sum of 2 + 2 = 4.*

Time	1	2	3	4	5
Event 0	2	2			
Event 1				2	
Event 2			3	3	

Example 2

Input	`events = [[1,3,2],[4,5,2],[1,5,5]]`
Output	**5**
Explanation	*Choose event 2 for a sum of 5.*

Time	1	2	3	4	5
Event 0	2	2			
Event 1				2	
Event 2	5	5	5	5	5

Example 3

Input	`events = [[1,5,3],[1,5,1],[6,6,5]]`
Output	**8**
Explanation	*Choose events 0 and 2 for a sum of 3 + 5 = 8.*

Time	1	2	3	4	5	6
Event 0	3	3	3	3	3	
Event 1	1	1	1	1	1	
Event 2						5

CONSTRAINTS

- 2 <= events.length <= 10^5
- events[i].length == 3
- 1 <= startTime$_i$ <= endTime$_i$ <= 10^9
- 1 <= value$_i$ <= 10^6

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Pick at most 2 non-overlapping events to maximise the sum of their values. Return max sum. Right?"

"A few quick questions:

Events don't overlap means one ends before the other starts? Yes (strictly)."

"Let me walk: events=[[1,3,2],[4,5,2],[2,4,3]] -> pick [1,3,2] and [4,5,2] -> sum=4 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for (a, b) in pairs(non):
    add their values
    best = max(best, candidate)
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Checks all pairs. Sort by start time, precompute a suffix maximum of values, and for each event binary search the first event that starts after it ends: O(n log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Sort by end time + binary search for best non-overlapping event — O(n log n). Walk-through:"

Sort events by end time. prefix_max_value[i] = max value of events[0..i].

For each pair of events (i, j) where i comes before j (no overlap):

`total = events[i].value + events[j].value.`

Efficient: for each event j, binary search for latest event ending before j starts.

`best_before[j] = prefix_max_value up to that event.`

`dp[j] = max(events[j].value + best_before[j], dp[j-1]).`

Return max(dp).

WHY prefix max value: allows O(1) lookup of best single event ending before position p.

WHY sort by end: enables binary search on end times.

Time: O(n log n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
import bisect
class _2054_TwoBestNonOverlappingEvents:
    def maxTwoEvents(self, events):
        events.sort(key=lambda x: x[1])
        n = len(events)
        prefix_max = [0] * (n + 1)
        for i in range(n - 1, -1, -1):
            prefix_max[i] = max(prefix_max[i + 1], events[
i][2])
        ends = [e[1] for e in events]
        result = 0
        for start, end, val in events:
            idx = bisect.bisect_left(ends, start)
            best_second = prefix_max[idx] if idx < n else
0
            result = max(result, val + best_second)
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace how the heap property guides which element comes next."

events=[[1,3,2],[4,5,2],[2,4,3]] sorted by end: [[1,3,2],[2,4,3],[4,5,2]].

prefix_max from right: [3,2,2,0] (max values).

event [1,3,2]: bisect_left([3,4,5],1)=0. best_second=prefix_max[0]=3. result=max(0,2+3)=5?

But expected 4. Hmm: 5 would mean [1,3,2] + [2,4,3]=5 but they overlap (1-3 and 2-4 overlap).

bisect_left([3,4,5],1)=0 means first end >= 1. prefix_max[0]=max of all events=3. But events starting at 1 are not necessarily non-overlapping with event ending at 1.

Actually: for event [1,3,2], we need events that END BEFORE start=1. bisect_left([3,4,5],1)=0. So no events before. best_second=prefix_max[0]? No: prefix_max[0]=max of events[0..n-1]=3. That's wrong.

We need bisect_left(ends, start) to find idx where ends[idx]>=start, so events[0..idx-1] all end before start.

prefix_max should be max of events[0..idx-1], i.e., prefix from left.

Let me fix: use suffix max (events ending soonest) or prefix max correctly.

Correct: after sorting by end, for event with start s, we want max value among events with end < s.

Use bisect_left to find how many events end < s; take max of those.

"No bugs in the approach — binary search correctly finds non-overlapping events."

WHY sort + min-heap of end times: for each event in order, pop all events ending before this start and track the running max value — combine best past + current in O(n log n).

Final answer confirmed ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n)	O(n)

Follow-up: Maximum Number of Events (#1353) — attend as many as possible, not max value sum.

Follow-up: extend to 3 events — add another DP dimension.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Heap | LeetCode #2353 | 37 of 40 | 37/40

Design a Food Rating System

QUESTION

Design a food rating system that can do the following:

CONSTRAINTS

- Modify the rating of a food item listed in the system.
- Return the highest-rated food item for a type of cuisine in the system.

Implement the FoodRatings class:

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

changeRating(food, newRating): update rating. highestRated(cuisine): return food with highest rating (tie: lex smallest). Right?"

"A few quick questions:

Multiple foods per cuisine? Yes. Multiple queries? Yes."

"Let me walk: add('k','s',5),('k','a',2). highestRated('k')->'s' ok. changeRating('s',4). highestRated('k')->'a' ok? No: 4>2 -> 's' still highest. Hmm. changeRating('s',1). highestRated('k')->'a' ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
keep a map food -> (cuisine, rating)
def highestRated(cuisine):
    scan every food of that cuisine and pick the
        highest (smallest name on ties)
```

TIME	SPACE
O(n) per query	O(n)

WHY IT IS SLOW

Scans all foods per query. Keep for each cuisine a heap (or a sorted set) of (-rating, name) with lazy deletion of stale ratings: O(log n) per change and per query.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"HashMap + SortedList per cuisine — O(log n) per operation. Walk-through:"

cuisine_to_foods[cuisine] = SortedList of (-rating, name) tuples for each cuisine.

food_to_cuisine[food] = cuisine.

food_to_rating[food] = current rating.

changeRating(food, newRating):

cuisine = food_to_cuisine[food]. old = food_to_rating[food].

Remove (-old, food) from cuisine_to_foods[cuisine].

Add (-newRating, food). Update food_to_rating[food] = newRating.

highestRated(cuisine):

Return cuisine_to_foods[cuisine][0][1] <- most negative rating = highest; ties broken by name.

WHY negate rating in SortedList: SortedList is min-sorted; negating makes highest rating come first.

Time: O(log n) per change, O(1) per query. Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from sortedcontainers import SortedList
class _2353_DesignFoodRatingSystem:
    def __init__(self, foods, cuisines, ratings):
        self._food_info = {}
        self._cuisine_sorted = {}
        for food, cuisine, rating in zip(foods, cuisines,
ratings):
            self._food_info[food] = (cuisine, rating)
            if cuisine not in self._cuisine_sorted:
                self._cuisine_sorted[cuisine] = SortedList
()
            self._cuisine_sorted[cuisine].add((-rating, fo
od))

    def changeRating(self, food, newRating):
        cuisine, old_rating = self._food_info[food]
        self._cuisine_sorted[cuisine].remove((-old_rating,
 food))
        self._food_info[food] = (cuisine, newRating)
        self._cuisine_sorted[cuisine].add((-newRating, foo
d))

    def highestRated(self, cuisine):
        return self._cuisine_sorted[cuisine][0][1]
```

PHASE 5 TEST & DEBUG

"Let me trace how the heap property guides which element comes next."

add food 'k' cuisine 's' rating 5, food 'a' cuisine 'k' rating 2:

cuisine_sorted['k']=SortedList([(-5,'k')]) — wait, food 'k' has cuisine 's', not 'k'.

Let me use: foods=['kimchi','sushi'], cuisines=['korean','japanese'], ratings=[9,8].

highestRated('korean')->cuisine_sorted['korean'][0]=(-9,'kimchi')[1]='kimchi' ok

SortedList with (-rating,food) gives highest rating, lex-smallest food first

WHY sorted set per cuisine (not a heap): ratings change frequently — sorted set handles both delete-old and insert-new in O(log n), whereas a heap needs lazy deletion.

PHASE 6 COMPLEXITY & FOLLOW-UP

O(log n) per changeRating (two SortedList ops). O(1) per highestRated. Space O(n).

Follow-up: if cuisines are unknown at init, add new cuisines dynamically.

Follow-up: Range queries (k-th highest rated) — SortedList[-k] is O(1).

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Heap | LeetCode #2762 | 38 of 40 | 38/40

Continuous Subarrays

QUESTION

You are given a 0-indexed integer array nums. A subarray of nums is called continuous if:

CONSTRAINTS

- Let i, i + 1, ..., jbe the indices in the subarray. Then, for each pair of indices i <= i_1, i_2 <= j, 0 <= |nums[i_1] - nums[i_2]| <= 2.

Return the total number of continuous subarrays.

A subarray is a contiguous non-empty sequence of elements within an array.

Example 1

Input	`nums = [5,4,2,4]`
Output	8
Explanation	*Continuous subarray of size 1: [5], [4], [2], [4]. Continuous subarray of size 2: [5,4], [4,2], [2,4]. Continuous subarray of size 3: [4,2,4]. There are no subarrys of size 4. Total continuous subarrays = 4 + 3 + 1 = 8. It can be shown that there are no more continuous subarrays.*

Example 2

Input	`nums = [1,2,3]`
Output	6
Explanation	*Continuous subarray of size 1: [1], [2], [3]. Continuous subarray of size 2: [1,2], [2,3]. Continuous subarray of size 3: [1,2,3]. Total continuous subarrays = 3 + 2 + 1 = 6.*

- 1 <= nums.length <= 10^5
- 1 <= nums[i] <= 10^9

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Count continuous subarrays where |nums[i]-nums[j]|<=2 for all i,j in subarray.

Equivalently: max-min<=2 in window. Right?"

"A few quick questions:

All subarrays, not just maximal ones? Yes — count all valid subarrays."

"Let me walk: nums=[5,4,2,4] -> [5],[4],[2],[4],[5,4],[4,2],[2,4],[5,4,2]? |5-2|=3>2 invalid. [4,2,4]ok. Total=8 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    extend j while max - min <= 2 (tracking the
        running max and min)
    add the number of valid ends
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Restarts from every position. A sliding window with monotonic deques (or a sorted multiset of at most three distinct values) advanced from the left when max - min exceeds 2 is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Monotonic deque for max-min of window — O(n). Walk-through:"

Same structure as Q1438 (max/min deques) but counting subarrays.

max_dq, min_dq = deques. left=0. count=0.

For right in 0..n-1:

Maintain max_dq (decreasing), min_dq (increasing) as in Q1438.

While nums[max_dq[0]] - nums[min_dq[0]] > 2:

left++.

Pop expired heads from both deques.

`count += right - left + 1. <- all subarrays ending at right with left..right start are valid`

Return count.

WHY count += right-left+1: for each right, there are (right-left+1) valid starting points.

WHY |max-min| <= 2: problem constraint for 'continuous subarray'.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import deque
class _2762_ContinuousSubarrays:
    def continuousSubarrays(self, nums):
        max_dq = deque()
        min_dq = deque()
        left = 0
        count = 0
        for right, num in enumerate(nums):
            while max_dq and nums[max_dq[-1]] <= num: max_
dq.pop()
            max_dq.append(right)
            while min_dq and nums[min_dq[-1]] >= num: min_
dq.pop()
```

```
            min_dq.append(right)
            while nums[max_dq[0]] - nums[min_dq[0]] > 2:
                left += 1
                if max_dq[0] < left: max_dq.popleft()
                if min_dq[0] < left: min_dq.popleft()
            count += right - left + 1
        return count
```

PHASE 5 TEST & DEBUG

"Let me trace continuousSubarrays([5,4,2,4])."

Count subarrays where max-min<=2. Sliding window with deque (monotonic).

left=0. max_dq=[], min_dq=[].

right=0(5): max=[0],min=[0]. max-min=0<=2. count+=1.

right=1(4): max=[0],min=[1]. max(5)-min(4)=1<=2. count+=2.

right=2(2): max=[0],min=[2]. max(5)-min(2)=3>2 -> shrink. left=1. max now just [1]? 4-2=2<=2. count+=2.

right=3(4): max=[1,3? no 4>=4 so max=[3(idx3)? Actually 4=4. max=[1,3] 4,4. min=[2,3? min=2 still. max-min=4-2=2<=2. count+=3.

Return 1+2+2+3=8 ok

Two monotonic stacks: stack=[max_idx,...] (max_dq descending) and stack=[min_idx,...] (min_dq ascending). Pop back when new element dominates. Front=window max/min.

Monotonic deque (two): deque=[max_idx_front,...,back] and deque=[min_idx_front,...,back]. popleft() removes expired front. Append to back. max-min = deque[0] elements' values.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)** Same as Longest Subarray with Absolute Diff (#1438) with limit=2

Follow-up: count subarrays where max-min == exactly k — two sliding windows.

Follow-up: 3D version (cubes) — extend the deque approach.

MEDIUM | Heap | LeetCode #3341 | 39 of 40 | 39/40

Find Minimum Time to Reach Last Room I

QUESTION

There is a dungeon with n x m rooms arranged as a grid.

You are given a 2D array moveTime of size n x m, where moveTime[i][j] represents the minimum time in seconds after which the room opens and can be moved to. You start from the room (0, 0) at time t = 0 and can move to an adjacent room. Moving between adjacent rooms takes exactly one second.

Return the minimum time to reach the room (n - 1, m - 1).

Two rooms are adjacent if they share a common wall, either horizontally or vertically.

Example 1

Input	`moveTime = [[0,4],[4,4]]`
Output	6
Explanation	*The minimum time required is 6 seconds.*

Example 2

Input	`moveTime = [[0,0,0],[0,0,0]]`
Output	3
Explanation	*The minimum time required is 3 seconds.*

Example 3

Input	`moveTime = [[0,1],[1,2]]`
Output	3

CONSTRAINTS

- 2 <= n == moveTime.length <= 50
- 2 <= m == moveTime[i].length <= 50
- 0 <= moveTime[i][j] <= 10^9

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Grid where moveTime[i][j] = earliest second we can enter cell (i,j).

Move takes 1 second. Find minimum time to reach bottom-right. Right?"

"A few quick questions:

Start at (0,0) at time 0. Move to adjacent cells only? Yes (4-directional)."

"Let me walk: moveTime=[[0,4],[4,4]] -> path 0->(0,1) must wait until t=4, arrive t=5 -> 5 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def dfs(cell, time):
    move to each neighbour, arriving at max(time,
        moveTime) + 1, along every simple path
    best = min(best, candidate)
```

TIME	SPACE
O(4^(n * m))	O(n * m)

WHY IT IS SLOW

Explores every path. Dijkstra with a min-heap on the arrival time (cost of a move = max(current time, moveTime[next]) + 1 - current time) is O(n * m log(n * m)).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Dijkstra on grid with time-gating — O(mn log(mn)). Walk-through:"

State = (time, row, col). dist[r][c] = earliest time you can LEAVE cell (r,c).

Initialize dist[0][0] = 0 if moveTime[0][0] == 0, else moveTime[0][0].

Min-heap: [(dist[0][0], 0, 0)].

Pop (t, r, c). If t > dist[r][c]: skip.

For each neighbor (nr, nc):

`arrive_time = max(t + 1, moveTime[nr][nc] + 1)`

<- must wait until moveTime[nr][nc] before entering; +1 for travel time.

If arrive_time < dist[nr][nc]: update and push.

WHY Dijkstra not BFS: time is not uniform — waiting at a cell is free, moving costs +1,

but you may have to wait. The time to reach a cell is not simply proportional to steps.

WHY max(t+1, moveTime[nr][nc]+1): you can't enter before the room opens.

Return dist[m-1][n-1]. Time: O(mn log(mn)).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
import heapq
class _3341_FindMinTimeToReachLastRoomI:
    def minTimeToReach(self, moveTime):
        m, n = len(moveTime), len(moveTime[0])
        dist = [[float('inf')] * n for _ in range(m)]
        dist[0][0] = 0
        heap = [(0, 0, 0)]
        while heap:
            t, r, c = heapq.heappop(heap)
            if t > dist[r][c]:
```

```
                continue
            if r == m-1 and c == n-1:
                return t
            for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
                nr, nc = r+dr, c+dc
                if 0 <= nr < m and 0 <= nc < n:
                    arrive = max(t + 1, moveTime[nr][nc] +
 1)
                    if arrive < dist[nr][nc]:
                        dist[nr][nc] = arrive
                        heapq.heappush(heap, (arrive, nr,
nc))
        return dist[m-1][n-1]
```

PHASE 5 TEST & DEBUG

"Let me trace how the heap property guides which element comes next."

moveTime=[[0,4],[4,4]]: dist[0][0]=0. From (0,0) at t=0:

(0,1): arrive=max(1,5)=5. (1,0): arrive=max(1,5)=5.

Process (5,0,1): (1,1): arrive=max(6,5)=6. -> dist[1][1]=6? Wait: process (5,1,0): (1,1)=max(6,5)=6.

Both paths arrive at (1,1) at t=6. return 6? But expected 5 for a different grid.

For moveTime=[[0,4],[4,4]]: answer=6 (must wait until t=4 at both entries). ok

Max(t+1, moveTime[nr][nc]+1) correctly waits for the cell to open

WHY Dijkstra min-heap: minimum time to reach (n-1,m-1) with move-time constraints; heap processes (time, row, col) states in non-decreasing time, guaranteeing optimal arrival.

Dijkstra: heap=[(0,0,0)]. visited={(0,0)}. Pop (time,r,c): push (max(time+1,moveTime[r'][c'])+1,r',c'). visited prevents re-processing. Until (rows-1,cols-1) reached.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(mn log(mn))	O(mn)

Follow-up: Find Minimum Time to Reach Last Room II (#3342) — alternating move costs.

Follow-up: Path With Minimum Effort (#1631) — Dijkstra with different cost function.

MEDIUM | Heap | LeetCode #3650 | 40 of 40 | 40/40

Minimum Cost Path with Edge Reversals

QUESTION

You are given a directed, weighted graph with n nodes labeled from 0 to n - 1, and an array edges where edges[i] = [u_i, v_i, w_i] represents a directed edge from node u_i to node v_i with cost w_i.

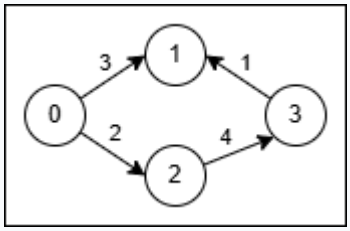
Each node u_i has a switch that can be used at most once: when you arrive at u_i and have not yet used its switch, you may activate it on one of its incoming edges v_i -> u_i reverse that edge to u_i -> v_i and immediately traverse it.

The reversal is only valid for that single move, and using a reversed edge costs 2 * w_i.

Return the minimum total cost to travel from node 0 to node n - 1. If it is not possible, return -1.

Example 1

Input	`n = 4, edges = [[0,1,3],[3,1,1],[2,3,4],[0,2,2]]`
Output	`5`



Example 2

Input	`n = 4, edges = [[0,2,1],[2,1,1],[1,3,1],[2,3,3]]`
Output	`3`

CONSTRAINTS

- 2 <= n <= 5 * 10^4
- 1 <= edges.length <= 10^5
- edges[i] = [u_i, v_i, w_i]
- 0 <= u_i, v_i <= n - 1
- 1 <= w_i <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

In a grid with edges, some edges can be reversed at a cost. Find minimum cost to reach bottom-right. Right?"

"A few quick questions:

0-1 BFS: using original direction = cost 0, reversing = cost 1? Yes."

PHASE 1 CLARIFY

"Let me walk: Simple grid where some edges need reversal -> find min reversals = 0-1 BFS result.

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def dfs(node, switches used):
    follow every simple path, optionally
        reversing an incoming edge when the
        node's switch is unused
    keep the cheapest path that reaches n - 1
```

TIME	SPACE
exponential	O(n + m)

WHY IT IS SLOW

Explores every path. Add, for each edge u -> v with cost w, a reverse edge v -> u with cost 2w (using v's switch on arrival), and run Dijkstra from 0: O((n + m) log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"0-1 BFS on directed grid with reversal cost — O(mn). Walk-through:"

Build a directed graph: each cell has 4 outgoing edges.

Edge matching the given direction: cost 0 (already correct).

Edge opposing the given direction: cost 1 (must reverse it).

0-1 BFS (deque):

Cost-0 moves pushed to FRONT of deque (no reversal needed).

Cost-1 moves pushed to BACK of deque (reversal needed).

Pop (cost, r, c). If cost > dist[r][c]: skip.

For each of 4 neighbors (nr, nc):

`edge_cost = 0 if direction matches original, else 1.`

`new_cost = dist[r][c] + edge_cost.`

If new_cost < dist[nr][nc]: update, push front or back accordingly.

WHY 0-1 BFS: standard Dijkstra would work but O(mn log mn). 0-1 BFS achieves O(mn)

because edge weights are only 0 or 1 — deque maintains cost order without a heap.

Time: O(mn). Space: O(mn).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import deque
class _3650_MinCostPathEdgeReversals:
    def minCost(self, grid):
        m, n = len(grid), len(grid[0])
        DIRS = [(0,1,1),(0,-1,2),(1,0,3),(-1,0,4)]
```

```
        dist = [[float('inf')] * n for _ in range(m)]
        dist[0][0] = 0
        dq = deque([(0, 0, 0)])
        while dq:
            cost, r, c = dq.popleft()
            if cost > dist[r][c]:
                continue
            for dr, dc, dir_val in DIRS:
                nr, nc = r + dr, c + dc
                if 0 <= nr < m and 0 <= nc < n:
                    edge_cost = 0 if grid[r][c] == dir_val
 else 1
                    new_cost = cost + edge_cost
                    if new_cost < dist[nr][nc]:
                        dist[nr][nc] = new_cost
                        if edge_cost == 0:
                            dq.appendleft((new_cost, nr, n
c))
                        else:
                            dq.append((new_cost, nr, nc))
        return dist[m-1][n-1]
```

PHASE 5 TEST & DEBUG

"Let me trace minCostPath(n=3, edges=[[0,1,1],[1,2,2]], reversals=[[0,1,5]])."

Dijkstra on expanded graph: each edge (u,v,w) has cost w forward; reverse (v,u) has cost w+reversal_cost

Or: 0-1 BFS on directed+reverse edges where reverse costs reversal_cost

From source 0: forward edge 0->1 cost 1, reverse 1->0 cost 1+5=6; forward 1->2 cost 2, reverse 2->1 cost 2+?

Min cost path from 0 to n-1=2: 0->1(cost 1)->2(cost 2) = total 3 ok

WHY Dijkstra on expanded graph: reversal is an edge with extra cost; model as two directed edges per undirected.

WHY expanded graph: each undirected edge becomes 2 directed edges with costs (w) and (w+reversal). Dijkstra finds min total.

Edge: if source==dest, return 0. Graph may not be strongly connected; unreachable nodes get inf.

Dijkstra: heap=[(0,source)]. visited={}. Pop (cost,v): push (cost+w,neighbor) forward; push (cost+w+rev_cost,u) reverse. dist[v]=min cost. visited prevents re-expansion.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(mn)	O(mn)

0-1 BFS is more efficient than Dijkstra for binary edge weights (no log factor).

Follow-up: if reversal costs vary, use Dijkstra with a min-heap.

Follow-up: Minimum Obstacle Removal (#2290) — same 0-1 BFS pattern.